

Машинное обучение:

NLP & RL

Переработанная расширенная методичка
для подготовки к экзамену

Не шпаргалка из списков, а связный конспект:
мотивация → формальная постановка → подробный
вывод формул → алгоритм → примеры →
экзаменационный ответ.

Содержание

Как устроена новая версия	2
Карта курса на одном листе	2
1 Bag-of-Words и TF-IDF	3
1.1 Bag-of-Words как первый способ превратить текст в вектор	3
1.2 Варианты признаков	4
1.3 Почему появляется TF-IDF	5
2 Word embeddings. Как обучается Word2vec?	7
2.1 Distributional hypothesis	7
2.2 CBOW и Skip-gram	9
2.3 Параметры Word2vec	9
2.4 Почему обучение дает смысловые векторы	11
3 Negative sampling в Word2vec	12
3.1 Проблема полного softmax	12
3.2 Objective	13
3.3 Откуда берутся негативы	13
4 Как работает recurrent block в RNN	15
4.1 Базовая рекуррентная ячейка	15
4.2 Задачи, которые решает RNN	16
4.3 Backpropagation Through Time	16
4.4 Почему RNN уступили Transformer в NLP	16
5 Attention mechanism	17
5.1 Query, Key, Value	17
5.2 Виды score-функций	17
5.3 Attention в seq2seq переводе	18
6 Machine translation. Encoder-decoder models	21
6.1 Вероятностная постановка	21
6.2 Encoder-decoder без attention	21
6.3 Encoder-decoder с attention	22
6.4 Teacher forcing и inference	22
6.5 Современная машинная трансляция	23
7 Как измерять качество в text generation	25
7.1 Почему задача оценки сложная	25
7.2 Likelihood и perplexity	25
7.3 Overlap-метрики	25
7.4 Semantic metrics и human evaluation	26
8 Unsupervised approach to machine translation	27
8.1 Что значит unsupervised MT	27
8.2 Denoising autoencoding	27
8.3 Back-translation	27
8.4 Инициализация и ограничения	28
9 Self-attention mechanism	29
9.1 Формальная запись	29
9.2 Маски	30

9.3 Multi-head attention	31
9.4 Порядок и сложность	31
10 Transformer architecture	32
10.1 Из чего состоит Transformer block	32
10.2 Decoder block	33
10.3 Зачем residual и LayerNorm	33
10.4 Encoder-only, decoder-only, encoder-decoder	33
11 BERT pretraining. MLM and NSP objectives	35
11.1 Вход BERT	35
11.2 Masked Language Modeling	35
11.3 Next Sentence Prediction	36
12 BERT fine-tuning. Transfer learning paradigm. Fine-tuning strategies	37
12.1 Transfer learning в NLP	37
12.2 Типовые heads	37
12.3 Стратегии fine-tuning	38
13 Contrastive learning paradigm. Examples	39
13.1 Позитивы и негативы	39
13.2 Почему нужны negatives и что такое collapse	41
13.3 Примеры	41
14 Как CLIP обучали для совместных embeddings текста и изображений	42
14.1 Данные и архитектура	42
14.2 Batch contrastive objective	42
14.3 Почему получается zero-shot classification	43
14.4 Ограничения	43
15 Reinforcement Learning: постановка задачи и отличие от Supervised Learning	44
15.1 Основные сущности RL	44
15.2 Чем RL отличается от supervised learning	45
16 Markov property	47
16.1 Определение	47
16.2 Markov Decision Process	47
16.3 Почему это важно	47
17 Reward discounting. Cumulative reward	49
17.1 Return	49
17.2 Зачем нужен discounting	50
17.3 Пример	51
18 Value function. Как вычислить для session?	52
18.1 Определение	52
18.2 Вычисление для одной session	52
18.3 Monte Carlo и TD	52
19 Q-function. Как научиться ее оценивать двумя способами?	54
19.1 Определение	54
19.2 Способ 1: Monte Carlo estimation	54
19.3 Способ 2: Bellman / TD learning	55
20 Как policy gradient оценивает градиент reward по параметрам policy	57

20.1 Objective	57
20.2 Likelihood ratio trick	57
20.3 Интуиция	58
21 Что такое baseline в policy gradient?	60
21.1 Формула с baseline	60
21.2 Почему baseline не вносит bias	60
21.3 Value baseline и advantage	61
22 Почему прямое обучение через likelihood / cross-entropy не всегда достаточно для language modeling	63
22.1 Что оптимизирует cross-entropy	63
22.2 Exposure bias	63
22.3 Token-level objective против sequence-level quality	63
22.4 One-to-many и mode covering	63
22.5 Почему здесь появляется RL	64
Финальная шпаргалка: связи между темами	66
Типовые устные вопросы	66
План подготовки на 3 дня	67
Приложение А. Сквозной пример: от сырого текста к модели	68
Приложение В. Размерности тензоров	70
Приложение С. Мини-выводы	72
Приложение D. Экзаменационные мини-задачи	74
Приложение Е. Подробные математические выводы	78
Последняя страница: 22 ответа в одну строку	88

Как устроена новая версия

Эта версия написана как методичка для устного экзамена, а не как набор коротких карточек. В каждом билете сначала объясняется, *зачем* появилась идея, затем дается аккуратная математическая запись, потом показывается схема работы метода и только после этого формулируется короткий ответ, который удобно произнести преподавателю. Такой порядок важен: на экзамене почти всегда спрашивают не только формулу, но и мотивацию, ограничения и связь с соседними темами.

В документе много рисунков. Они не заменяют формулы, а помогают держать в голове вычислительный граф: где появляются признаки, что именно обучается, какие величины являются вероятностями, где находится bottleneck, почему attention и RL-градиенты устроены именно так.

Интуиция

Хороший ответ на любой пункт программы обычно состоит из четырех частей: постановка задачи, модель или алгоритм, функция потерь/критерий, ограничения метода. Если вы умеете для каждого раздела быстро заполнить эти четыре пункта, билет почти всегда получается связным.

Карта курса на одном листе

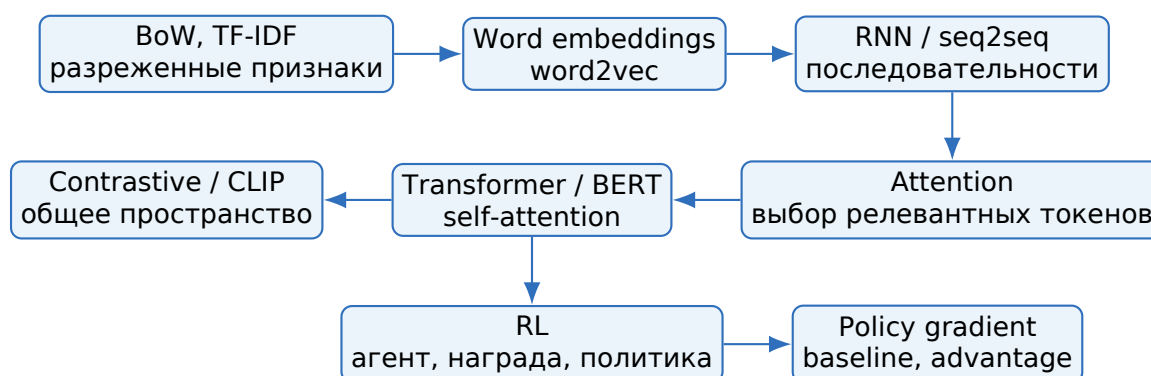


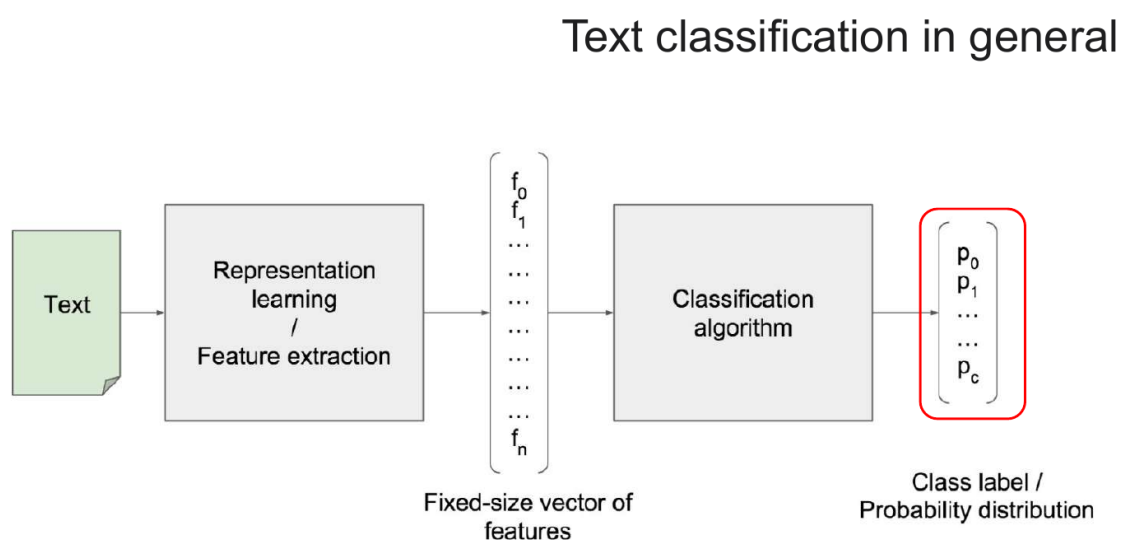
Рис. 1: Линия курса: от ручных текстовых признаков к большим предобученным представлениям и оптимизации последовательностей через RL.

1. Bag-of-Words и TF-IDF

Интуиция

Представьте, что нужно классифицировать новости по темам. Если в тексте часто встречаются слова “футбол”, “матч”, “гол”, то документ, скорее всего, про спорт. Для простого тематического анализа не всегда нужно понимать синтаксис: достаточно посчитать, какие слова и фразы встречаются. Так появляется Bag-of-Words.

1.1 Bag-of-Words как первый способ превратить текст в вектор



7

Рис. 2: Общая схема текстовой классификации из презентаций. Сначала текст превращается в фиксированный вектор признаков, затем этот вектор подается в классификатор. Именно поэтому BoW, TF-IDF, embeddings и Transformer-представления в курсе можно рассматривать как разные варианты блока *representation learning / feature extraction*.

Bag-of-Words

the dog is on the table

0	0	1	1	0	1	1	2
are	cat	dog	is	now	on	table	the

- Problems:
 - No information about words order
 - Word vectors are huge and very sparse
 - Word vectors are not normalized
 - Same words can take different forms

12

Рис. 3: Bag-of-Words в презентациях. Слайд показывает главное ограничение BoW: порядок слов теряется, векторы получаются огромными и разреженными, а разные формы одного слова считаются разными признаками без нормализации.

Модель машинного обучения обычно принимает на вход числовой вектор фиксированной длины. Текст же является последовательностью слов переменной длины. Bag-of-Words решает эту проблему максимально прямолинейно: строим словарь $\mathcal{V}$ по корпусу и для каждого документа считаем, сколько раз встретился каждый токен из словаря.

Пусть $\mathcal{V} = \{t_1, \dots, t_m\}$. Тогда документ d превращается в вектор

$$x(d) = (c(t_1, d), c(t_2, d), \dots, c(t_m, d)) \in \mathbb{R}^m,$$

где $c(t_i, d)$ - число вхождений токена t_i в документ. Вектор получается очень длинным и разреженным: словарь может содержать десятки тысяч слов, но в одном документе встречается малая часть из них.

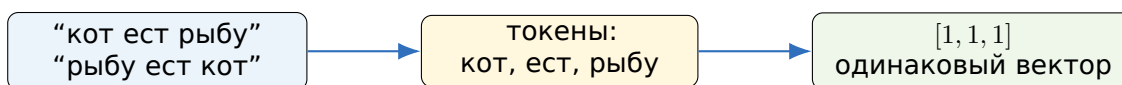


Рис. 4: BoW игнорирует порядок слов, поэтому разные предложения могут стать одинаковыми векторами.

Главное достоинство BoW - простота. Его легко объяснить, легко реализовать, он быстро обучается и часто дает сильный baseline для классификации текстов, спам-фильтрации и информационного поиска. Главный недостаток - потеря порядка и семантики: слова "машина" и "автомобиль" являются разными координатами, даже если смысл близок.

1.2 Варианты признаков

В бинарном BoW координата равна единице, если слово встретилось хотя бы раз. В count BoW координата равна числу вхождений. В n-gram BoW признаками становятся не только слова, но и фразы из n подряд идущих токенов. Например, биграмма

“not good” гораздо информативнее, чем отдельные слова “not” и “good”, потому что она сохраняет часть локального порядка.

На практике BoW почти всегда сопровождается предобработкой: токенизация, приведение к нижнему регистру, удаление пунктуации, иногда лемматизация или стемминг, фильтрация слишком редких и слишком частых токенов. Эти шаги являются частью модели: если обучить векторизатор на одних правилах, а применять на других, качество может резко ухудшиться.

1.3 Почему появляется TF-IDF

TF-IDF

- **Term Frequency (tf):** gives us the frequency of the word in each document in the corpus.

$$\text{tf}(t, d) = f_{t,d}$$

- **Inverse Document Frequency (idf):** used to calculate the weight of rare words across all documents in the corpus. The words that occur rarely in the corpus have a high IDF score.

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

N : total number of documents in the corpus $N = |D|$

$|\{d \in D : t \in d\}|$: number of documents where the term t appears 28

Рис. 5: TF-IDF из презентаций. Term Frequency отвечает за частоту слова внутри документа, а Inverse Document Frequency штрафует слова, встречающиеся во многих документах корпуса. Поэтому TF-IDF выделяет слова, специфичные для документа, а не просто частые в языке.

Обычный счетчик переоценивает частые служебные слова. Слово “и” может встретиться в документе много раз, но почти ничего не говорит о теме. Слово “квантование” встречается реже, зато сильно характеризует документ. TF-IDF вводит идею: слово важно, если оно часто встречается в данном документе, но редко встречается в корпусе в целом.

Формулы и определения

Term Frequency:

$$\text{tf}(t, d) = c(t, d) \quad \text{или} \quad \text{tf}(t, d) = \frac{c(t, d)}{\sum_{t'} c(t', d)}$$

Inverse Document Frequency:

$$\text{idf}(t) = \log \frac{N + 1}{\text{df}(t) + 1} + 1,$$

где N - число документов, $\text{df}(t)$ - число документов, в которых встретился токен t . Тогда

$$\text{tfidf}(t, d) = \text{tf}(t, d) \text{idf}(t).$$

Геометрически каждый документ становится точкой в пространстве слов. Для поиска похожих документов часто используют косинусное сходство

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}.$$

Если нормировать TF-IDF-векторы по L_2 -норме, сравнивается не длина документа, а направление: какие слова относительно важны.

Полезное замечание

TF-IDF не “понимает смысл”. Он только использует статистику встречаемости. Но эта статистика часто настолько информативна, что линейная модель поверх TF-IDF бывает очень сильным baseline и иногда побеждает более сложные модели на малых данных.

Как отвечать на экзамене

BoW представляет документ как вектор частот слов из словаря, игнорируя порядок. TF-IDF улучшает BoW: вес слова равен произведению частоты слова в документе и обратной документной частоты в корпусе. Частые во всех документах слова получают маленький вес, специфичные для документа слова - большой. Метод прост, интерпретируем и полезен как baseline, но не кодирует семантическую близость и почти не учитывает порядок слов.

Частые ошибки

Не надо говорить, что IDF считается по числу всех токенов. Он считается по числу документов, где слово встретилось. Также нельзя смешивать TF-IDF и embeddings: TF-IDF - разреженный вектор документа, а embedding обычно плотный вектор слова, токена или предложения.

2. Word embeddings. Как обучается Word2vec?

Интуиция

В BoW каждое слово - отдельная координата. Поэтому “кошка” и “котенок” почти так же далеки, как “кошка” и “процессор”. Word embeddings исправляют это: слово становится точкой в плотном пространстве, а смысловая близость отражается геометрической близостью.

2.1 Distributional hypothesis

One-hot vectors

- One-hot vectors:

Rome = [1, 0, 0, 0, 0, 0, ..., 0]

Paris = [0, 1, 0, 0, 0, 0, ..., 0]

Italy = [0, 0, 1, 0, 0, 0, ..., 0]

France = [0, 0, 0, 1, 0, 0, ..., 0]

Problems:

- Huge vectors
- VERY sparse
- No semantics or word similarity information included

35

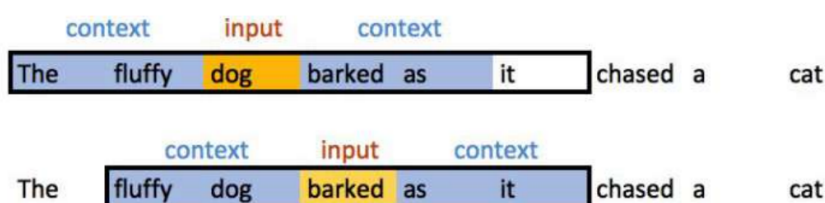
Рис. 6: One-hot vectors. One-hot кодировка делает все разные слова ортогональными. Из-за этого в ней нет семантической близости: слова *Rome* и *Paris* находятся так же далеко друг от друга, как *Rome* и любое неродственное слово.

Distributional semantics

Does vector similarity imply semantic similarity?

“You shall know a word by the company it keeps”

Firth, 1957

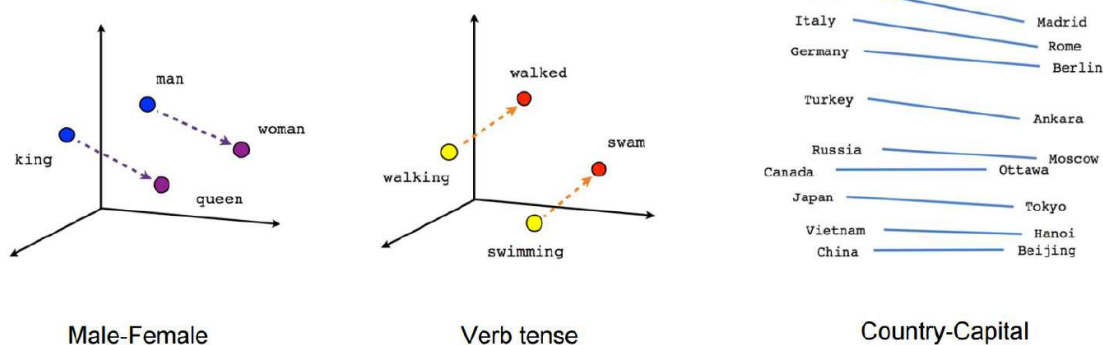


36

Рис. 7: Distributional semantics. Идея Firth: слово узнается по его окружению. Word2vec и похожие методы используют это как обучающий сигнал: похожие контексты должны давать похожие векторы.

Word2vec

- **Word2vec** (Mikolov et al. 2013) - a framework for learning word embeddings



44

Рис. 8: Word2vec и аналогии. В презентациях embeddings иллюстрируются линейными отношениями вроде male-female, verb tense, country-capital. Это не строгая логика, а эмпирический эффект геометрии векторов, обученных на контекстах.

Классическая гипотеза в NLP звучит так: слово определяется контекстами, в которых оно встречается. Если два слова часто появляются рядом с похожими словами, их значения, вероятно, близки. Word2vec не получает словарных определений и не знает разметки. Он решает self-supervised задачу, автоматически получая обучающие пары из сырого текста.

Пусть есть предложение:

I like machine learning very much.

Если размер окна равен 2, то для центрального слова “machine” контекстом будут “I”, “like”, “learning”, “very”. Из текста получаются пары слово-контекст, и модель учится делать реальные пары вероятными.

2.2 CBOW и Skip-gram

У Word2vec есть две базовые архитектуры. CBOW предсказывает центральное слово по контексту. Skip-gram делает наоборот: по центральному слову предсказывает соседние слова. CBOW быстрее и сглаживает контексты, Skip-gram обычно лучше работает с редкими словами, потому что каждое центральное слово порождает несколько обучающих пар.

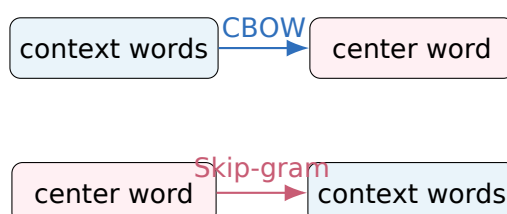
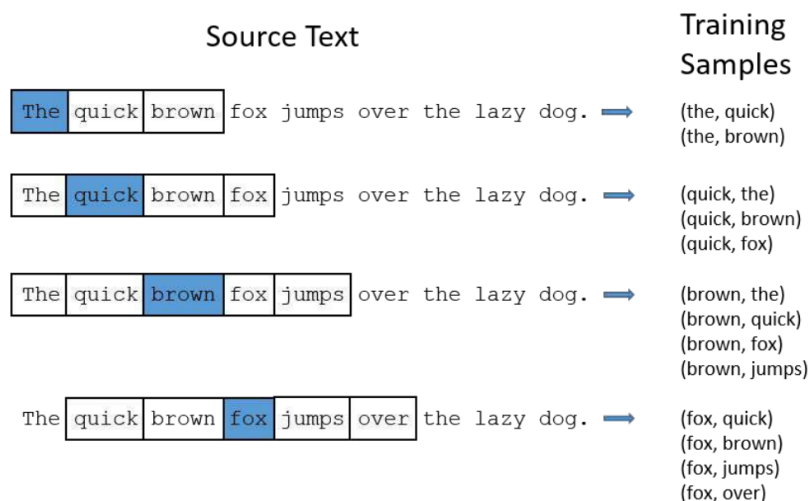


Рис. 9: CBOW и Skip-gram решают обратные self-supervised задачи.

2.3 Параметры Word2vec

Embeddings: word2vec

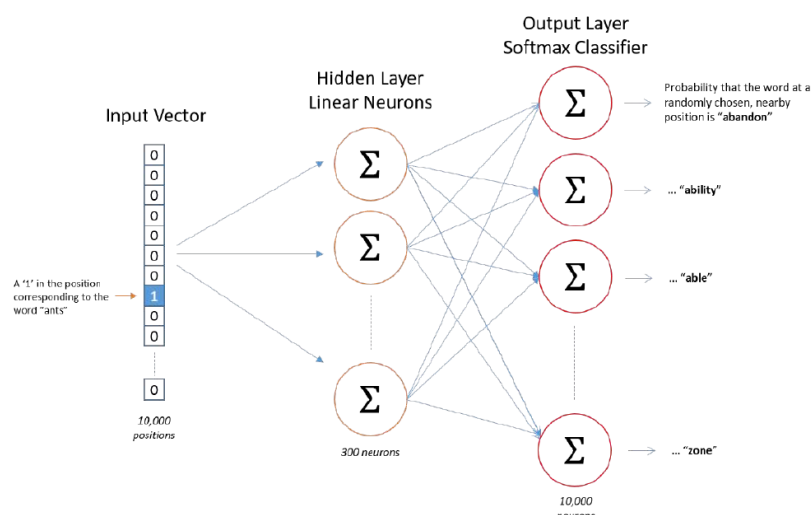


Source: <http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

45

Рис. 10: Формирование skip-gram пар. Из скользящего окна строятся пары *center word* - *outside word*. Например, для центрального слова модель получает несколько контекстных слов слева и справа, и каждая такая пара становится обучающим примером.

Embeddings: word2vec



Source: <http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

48

Рис. 11: Архитектура word2vec. One-hot вход выбирает строку/столбец embedding-матрицы. После обучения веса скрытого слоя фактически становятся lookup table для словарных embeddings.

Word2vec: two models

Continuous BOW (CBOW)

$$p(w_i | w_{i-h}, \dots, w_{i+h})$$

Predict center word from
(bag of) context words

- Predicting one word each time
- Relatively fast

Skip-gram

$$p(w_{i-h}, \dots, w_{i+h} | w_i)$$

Predict context ("outside")
words (position independent)
given center word

- Predicting context by one word
- Much slower
- Better with infrequent words

54

Рис. 12: CBOW и Skip-gram. CBOW предсказывает центральное слово по контексту и обычно быстрее. Skip-gram предсказывает контекст по центральному слову, медленнее, но лучше работает с редкими словами.

Пусть $|\mathcal{V}|$ - размер словаря, d - размерность embedding. У модели есть две матрицы:

$$W_{in} \in \mathbb{R}^{|\mathcal{V}| \times d}, \quad W_{out} \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

Вектор центрального слова w берется из W_{in} и обозначается v_w . Вектор контекстного слова c берется из W_{out} и обозначается u_c . Скалярное произведение $u_c^\top v_w$ показывает, насколько слово c похоже на правдоподобный контекст для w .

Для Skip-gram с полным softmax вероятность контекстного слова равна

$$P(c|w) = \frac{\exp(u_c^\top v_w)}{\sum_{j=1}^{|\mathcal{V}|} \exp(u_j^\top v_w)}.$$

Loss для наблюдаемых пар:

$$\mathcal{L} = - \sum_{(w,c) \in D} \log P(c|w).$$

2.4 Почему обучение дает смысловые векторы

Модель не обучается на задаче “выучи смысл”. Она обучается предсказывать контексты. Но для хорошего предсказания ей выгодно расположить слова с похожими контекстами рядом. Если “cat” и “kitten” часто окружены похожими словами, то их векторы начинают играть похожую роль в скалярных произведениях. Так возникает семантическая геометрия.

Полезное замечание

Известный пример $king - man + woman \approx queen$ - полезная интуиция, но не закон природы. Линейные аналогии зависят от корпуса, размерности, обучения и метода оценки.

Как отвечать на экзамене

Word2vec обучает плотные векторы слов на неразмеченном тексте. В Skip-gram модель по центральному слову предсказывает слова из контекстного окна; в CBOW по контексту предсказывает центральное слово. Для Skip-gram вероятность контекста задается softmax по словарю через скалярные произведения input- и output-embeddings. Обучение максимизирует вероятность реальных пар слово-контекст. На практике полный softmax дорогой, поэтому используют negative sampling или hierarchical softmax.

Частые ошибки

Не путайте Word2vec с supervised classification: разметка не нужна, пары создаются из текста автоматически. Еще одна частая ошибка - забыть, что в Word2vec две матрицы embeddings, хотя после обучения часто используют только input embeddings.

3. Negative sampling в Word2vec

Интуиция

Полный softmax спрашивает: “какое из сотен тысяч слов словаря является правильным контекстом?” Negative sampling задает более дешевый вопрос: “эта конкретная пара слово-контекст настоящая или случайная?”

3.1 Проблема полного softmax

Embeddings: negative sampling

Negative Sampling idea: only few words error is computed.
All other words have zero error, so no updates by the
backprop mechanism.

More frequent words are selected to be negative samples
more often. The probability for a selecting a word is just it's
weight divided by the sum of weights for all words.

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=0}^n (f(w_j)^{3/4})}$$

53

Рис. 13: Negative sampling в презентациях. Главная вычислительная идея: для одного положительного примера обновлять не все слова словаря, а только положительный контекст и несколько негативных слов, выбранных из noise distribution.

В знаменателе softmax нужно просуммировать по всему словарю. При словаре в сотни тысяч слов это означает, что для одной обучающей пары приходится трогать почти все output embeddings. Для больших корпусов это слишком дорого.

Negative sampling заменяет многоклассовую классификацию на бинарную. Для реальной пары (w, c) ставим label 1. Затем выбираем k случайных слов $n_1, \dots, n_k$ из noise distribution и считаем пары (w, n_i) отрицательными примерами с label 0.

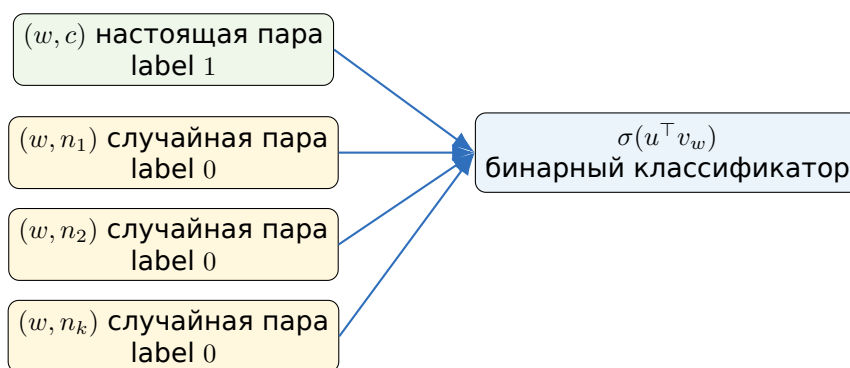


Рис. 14: Negative sampling превращает предсказание слова из словаря в несколько бинарных решений.

3.2 Objective

Для одной положительной пары (w, c) максимизируется

$$\log \sigma(u_c^\top v_w) + \sum_{i=1}^k \mathbb{E}_{n_i \sim P_n} \log \sigma(-u_{n_i}^\top v_w),$$

где $\sigma(z) = 1/(1 + e^{-z})$. Если записывать loss для минимизации:

$$\mathcal{L}_{NS} = -\log \sigma(u_c^\top v_w) - \sum_{i=1}^k \log \sigma(-u_{n_i}^\top v_w).$$

Первый член делает скалярное произведение настоящей пары большим. Второй делает скалярные произведения случайных пар маленькими.

3.3 Откуда берутся негативы

В классическом Word2vec негативные слова выбираются не равномерно, а из распределения

$$P_n(w) \propto f(w)^{3/4},$$

где $f(w)$ - частота слова. Степень $3/4$ сглаживает распределение: частые слова все еще выбираются чаще, но не настолько доминируют, как при чистом unigram distribution.

Формулы и определения

Стоимость полного softmax на один пример - $O(|V|)$. Стоимость negative sampling - $O(k)$, где k обычно порядка нескольких единиц или десятков. Обновляются только embedding центрального слова, embedding положительного контекста и embeddings выбранных негативов.

Как отвечать на экзамене

Negative sampling - это способ ускорить Word2vec. Вместо полного softmax по всему словарю модель решает бинарную задачу: настоящая ли пара слово-контекст. Для каждой положительной пары выбираются k негативных слов из noise distribution, часто $f(w)^{3/4}$. Loss увеличивает $u_c^\top v_w$ для реального контекста и уменьшает $u_n^\top v_w$ для случайных контекстов. Вычислительная стоимость пада-

ет с $O(|\mathcal{V}|)$ до $O(k)$.

Частые ошибки

Negative не означает “слово с отрицательной окраской”. Это просто отрицательный пример для бинарной классификации. Также negative sampling не является точным полным softmax; это другая, но очень полезная objective для обучения embeddings.

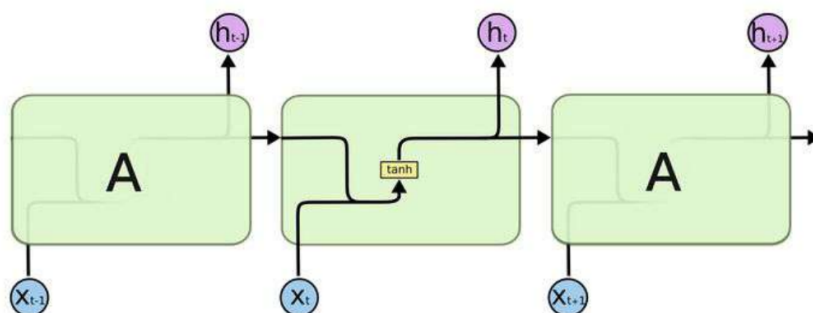
4. Как работает recurrent block в RNN

Интуиция

Текст - последовательность. Чтобы понять очередное слово, нужно помнить предыдущий контекст. RNN делает это через скрытое состояние: на каждом шаге она читает новый токен и обновляет память.

4.1 Базовая рекуррентная ячейка

Recap: Vanilla RNN



Source: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

16

Рис. 15: Vanilla RNN recap. RNN можно представить как одну и ту же ячейку, применяемую к каждому токenu. Скрытое состояние переносит информацию из прошлого, а общие веса позволяют обрабатывать последовательности переменной длины.

Пусть входная последовательность состоит из embeddings $x_1, \dots, x_T$. Vanilla RNN поддерживает hidden state h_t :

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b),$$

где ϕ - нелинейность, обычно $\tanh$ или ReLU . Если нужно выдавать предсказание на каждом шаге,

$$y_t = g(W_y h_t + c).$$

Одинаковые параметры W_x, W_h, b применяются на всех временных шагах. Поэтому RNN можно использовать для последовательностей разной длины.

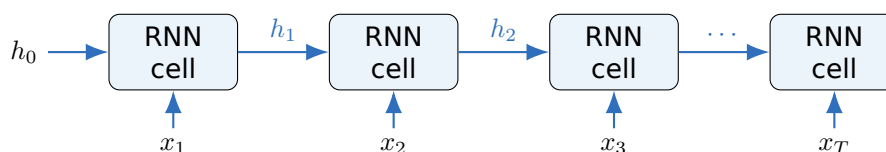


Рис. 16: RNN как одна и та же ячейка, развернутая во времени.

4.2 Задачи, которые решает RNN

RNN подходит для нескольких схем. В many-to-one вся последовательность сворачивается в последний hidden state, например для sentiment classification. В many-to-many на каждом шаге нужен выход, например для POS tagging. В seq2seq encoder читает входную последовательность, а decoder генерирует выходную.

Важно понимать, что h_t - не магическая память всего прошлого. Это вектор фиксированного размера. Вся информация о первых токенах должна пройти через цепочку обновлений до момента t , поэтому дальние зависимости трудно сохранять.

4.3 Backpropagation Through Time

Чтобы обучить RNN, вычислительный граф разворачивают на T шагов и применяют backpropagation через время. При этом градиенты многократно умножаются на матрицы, связанные с W_h и производными нелинейностей. Из-за этого возможны две проблемы:

- **vanishing gradients**: градиент к ранним шагам становится почти нулевым, модель плохо учит дальние зависимости;
- **exploding gradients**: градиент становится слишком большим, обучение нестабильно.

Для exploding gradients часто помогает gradient clipping. Для vanishing gradients были предложены LSTM и GRU, которые используют gates и более устойчивый путь передачи памяти.

4.4 Почему RNN уступили Transformer в NLP

Главный недостаток RNN - последовательность вычислений. Нельзя посчитать h_t , пока не посчитан h_{t-1} . Это плохо параллелится на GPU. Кроме того, путь информации от первого токена к последнему имеет длину $O(T)$. В self-attention каждый токен может напрямую обратиться к любому другому за один слой, хотя цена - квадратичная матрица attention.

Как отвечать на экзамене

Recurrent block на шаге t берет текущий вход x_t и предыдущее скрытое состояние h_{t-1} , применяет линейное преобразование и нелинейность, получая h_t . Параметры ячейки общие для всех временных шагов. Обучение идет через backpropagation through time. Основные проблемы vanilla RNN - затухающие/взрывающиеся градиенты и слабая параллелизация. LSTM и GRU добавляют gates, чтобы лучше контролировать память.

Частые ошибки

Не говорите, что RNN просто “запоминает все слова”. Она сжимает историю в fixed-size state. Также не забывайте, что hidden state и output - разные величины: output может строиться из hidden state, но не обязан совпадать с ним.

5. Attention mechanism

Интуиция

Если decoder переводит слово, ему не обязательно помнить все исходное предложение в одном векторе. Он может каждый раз выбирать, на какие входные токены смотреть. Attention - это дифференцируемый механизм такого выбора.

5.1 Query, Key, Value

Attention удобно описывать метафорой поиска в памяти. Query - это запрос: какая информация сейчас нужна. Key - адрес или описание элемента памяти. Value - содержимое, которое будет взято, если элемент оказался релевантным.

Для query q , keys k_i и values v_i сначала считаются scores:

$$e_i = \text{score}(q, k_i).$$

Затем scores нормируются softmax:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}.$$

Итоговый context vector - взвешенная сумма values:

$$c = \sum_i \alpha_i v_i.$$

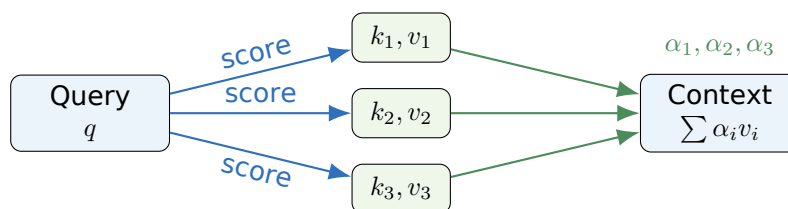


Рис. 17: Attention сравнивает query с keys и смешивает values с полученными весами.

5.2 Виды score-функций

В additive attention, популяризованном в ранних seq2seq моделях, score имеет вид

$$e_i = v_a^\top \tanh(W_q q + W_k k_i).$$

В dot-product attention:

$$e_i = q^\top k_i.$$

В Transformer используется scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Деление на $\sqrt{d_k}$ нужно для стабилизации softmax: при больших размерностях dot products имеют большую дисперсию, softmax может становиться слишком резким, и градиенты ухудшаются.

5.3 Attention в seq2seq переводе

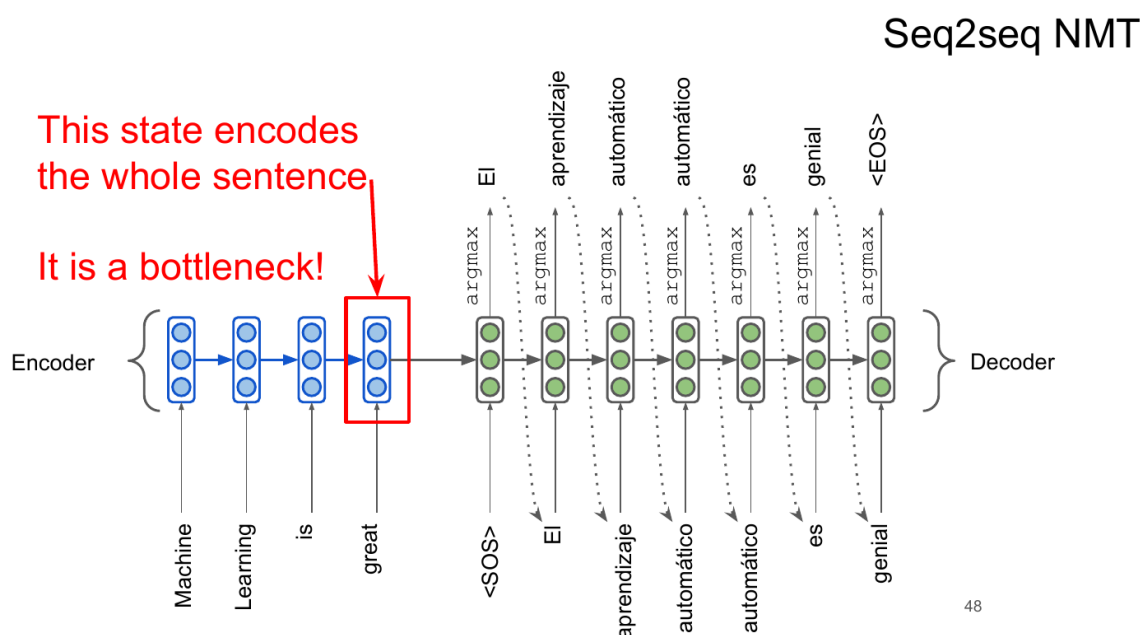


Рис. 18: Bottleneck в seq2seq. Без attention все исходное предложение должно быть закодировано в одном состоянии encoder. На слайде это прямо отмечено как bottleneck.

Attention

Main idea:

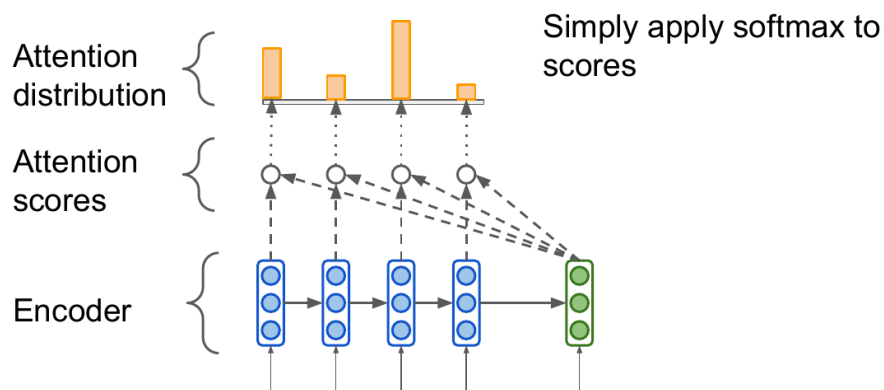
on each step of the **decoder**, use **direct connection to the encoder** to focus on a particular part of the source sequence



49

Рис. 19: Главная идея attention. На каждом шаге decoder получает прямую связь со всеми состояниями encoder и может сфокусироваться на нужной части source sequence.

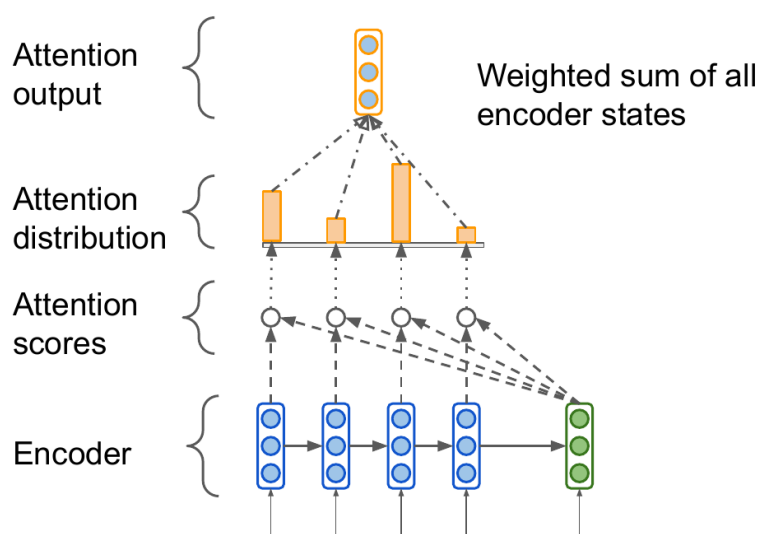
Seq2seq with attention



51

Рис. 20: Attention distribution. Scores превращаются в распределение attention через softmax. Поэтому веса не просто произвольные числа: они неотрицательны и суммируются в единицу.

Seq2seq with attention



52

Рис. 21: Attention output. Context vector является weighted sum всех encoder states. Это важная математическая деталь: attention не выбирает один токен жестко, а строит дифференцируемую смесь.

Attention in equations

Denote encoder hidden states $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^k$
and decoder hidden state at time step t $\mathbf{s}_t \in \mathbb{R}^k$

The attention scores $\mathbf{e}^t$ can be computed as dot product

$$\mathbf{e}^t = [\mathbf{s}_t^T \mathbf{h}_1, \dots, \mathbf{s}_t^T \mathbf{h}_N]$$

Then the attention vector is a linear combination of encoder states

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^k, \text{ where } \alpha_t = \text{softmax}(\mathbf{e}_t)$$

57

Рис. 22: Attention в формулах. Презентация записывает encoder hidden states, decoder state на шаге t , scores, softmax-веса и итоговый attention vector как линейную комбинацию encoder states.

Encoder строит состояния $h_1, \dots, h_n$ для source sentence. Decoder на шаге t имеет состояние s_t . Чтобы сгенерировать очередной target token, decoder считает

$$\alpha_{t,i} = \text{softmax}_i(\text{score}(s_t, h_i)), \quad c_t = \sum_i \alpha_{t,i} h_i.$$

Контекст c_t показывает, какие части source sentence сейчас полезны. Когда модель переводит “black”, attention должен концентрироваться на “черный”/“black”-соответствии; когда переводит существительное, веса смещаются к другому слову. Это не обязано быть строгим alignment один-к-одному, но часто дает понятную визуализацию.

Полезное замечание

Attention weights удобно интерпретировать, но осторожно: большой вес не всегда является строгим причинным объяснением. Attention показывает, как в данном слое смешиваются value-векторы, а не полную причинную цепочку решения модели.

Как отвечать на экзамене

Attention строит context vector как weighted sum values. Веса получаются сравнением query с keys и softmax-нормировкой. В машинном переводе query обычно связан с состоянием decoder, а keys/values - с состояниями encoder. Это снимает bottleneck одного fixed-size vector: decoder на каждом шаге может выбирать релевантные части входа.

Частые ошибки

Не путайте attention и self-attention. В encoder-decoder attention query может приходить из decoder, а keys/values из encoder. В self-attention все три набора строятся из одной последовательности.

6. Machine translation. Encoder-decoder models

Интуиция

Перевод - это не классификация предложения в один класс, а генерация последовательности переменной длины. Поэтому модель должна прочитать source sentence и затем по одному токenu породить target sentence.

6.1 Вероятностная постановка

Пусть исходное предложение $x = (x_1, \dots, x_n)$, перевод $y = (y_1, \dots, y_m)$. Модель задает условную вероятность

$$p(y|x) = \prod_{t=1}^m p(y_t | y_{<t}, x).$$

Это autoregressive factorization: каждый следующий token зависит от source и уже сгенерированного prefix.

6.2 Encoder-decoder без attention

Seq2seq NMT

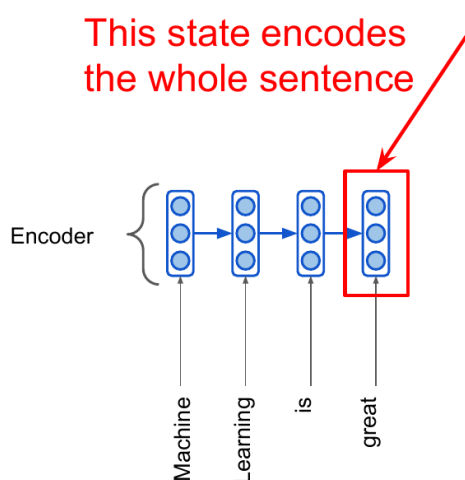


Рис. 23: Seq2seq encoder. Encoder читает исходную последовательность и сжимает ее в состояние, которое затем используется decoder. Именно это сжатие в один вектор становится bottleneck для длинных предложений.

В раннем RNN seq2seq encoder сжимал вход в один final state:

$$h_n = \text{Encoder}(x_1, \dots, x_n).$$

Decoder начинал с этого состояния и генерировал target tokens от <BOS> до <EOS>. Проблема очевидна: один вектор должен вместить все исходное предложение. Для длинных фраз это bottleneck.

6.3 Encoder-decoder с attention

С attention encoder возвращает всю последовательность состояний $H = (h_1, \dots, h_n)$. Decoder на шаге t строит context vector c_t как attention над H и предсказывает

$$p(y_t | y_{<t}, x) = \text{softmax}(W[s_t; c_t] + b).$$

Теперь информация из каждого source token доступна напрямую.

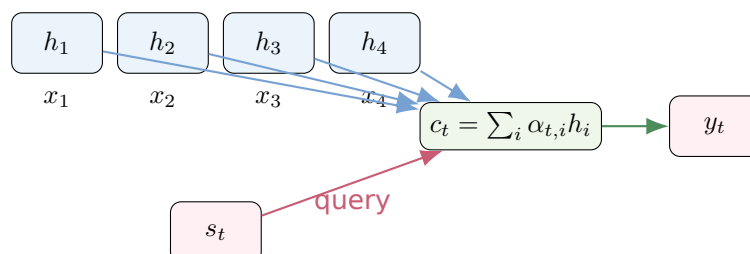


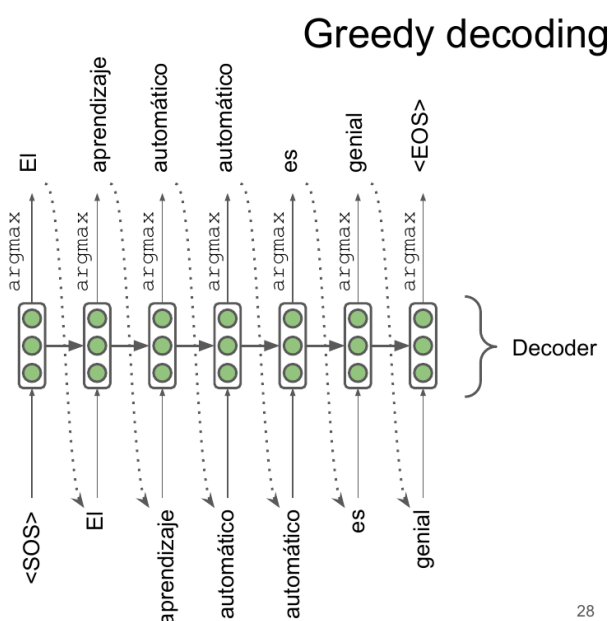
Рис. 24: Decoder на каждом шаге выбирает разные encoder states через attention.

6.4 Teacher forcing и inference

- Decoder predicts the most probable token (argmax) on each step
- The approach is **greedy**

Any problems with it?

Any mistake is treated as input on the next step!



28

Рис. 25: Greedy decoding. При greedy decoding decoder на каждом шаге выбирает самый вероятный токен. Проблема в том, что ошибка на раннем шаге становится входом на следующем шаге и может испортить всю генерацию.

Beam search

- On each step of decoder, keep track of the k most probable partial translations (which we call hypotheses)
- k is the beam size (in practice around 5 to 10)
- A hypothesis has a score which is its log probability:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- We search for high-scoring hypotheses, tracking top k on each step
- Beam search does not guarantee finding optimal solution

30

Рис. 26: Beam search. Beam search хранит несколько лучших частичных гипотез. Это компромисс между жадным decoding и невозможным полным перебором всех последовательностей.

При обучении decoder часто получает на вход настоящий предыдущий токен y_{t-1} , а не свой сгенерированный. Это называется teacher forcing. Loss:

$$\mathcal{L} = - \sum_{t=1}^m \log p_{\theta}(y_t | y_{<t}, x).$$

На inference настоящего предыдущего токена нет, поэтому модель кормится своими предсказаниями. Отсюда возникает exposure bias: маленькая ошибка в начале может привести к грефх, которого модель почти не видела при обучении.

Greedy decoding выбирает самый вероятный токен на каждом шаге. Beam search хранит несколько лучших partial hypotheses и часто дает более качественные переводы, но требует length normalization, иначе короткие последовательности могут получать преимущество из-за произведения вероятностей.

6.5 Современная машинная трансляция

Современная МТ обычно использует subword tokenization, Transformer encoder-decoder, cross-entropy training, label smoothing, beam search или sampling, а качество измеряется не одной метрикой, а набором automatic metrics и человеческой оценкой. Важно понимать, что encoder-decoder - это архитектурный принцип, а не обязательно RNN: Transformer тоже может быть encoder-decoder.

Как отвечать на экзамене

Machine translation формулируется как моделирование $p(y | x) = \prod_t p(y_t | y_{<t}, x)$. Encoder строит представления source sequence, decoder autoregressively генерирует target sequence. В старом seq2seq весь source сжимался в один вектор, что создавало bottleneck. Attention позволяет decoder на каждом шаге обращаться к разным encoder states. Обучение обычно идет через teacher forcing и cross-entropy, inference - через greedy decoding или beam search.

Частые ошибки

Не забывайте различать training и inference. При training часто используется teacher forcing, а при inference модель видит собственные прошлые токены. Также не надо говорить, что encoder-decoder всегда RNN: Transformer encoder-decoder - самый важный современный пример.

7. Как измерять качество в text generation

Интуиция

В классификации часто есть один правильный label. В генерации текста хороших ответов может быть много. Поэтому качество нельзя свести к точному совпадению строки с эталоном.

7.1 Почему задача оценки сложная

Для одного исходного предложения может существовать несколько хороших переводов. Для одного вопроса может быть несколько корректных ответов. Хороший текст должен быть грамматичным, релевантным, содержательно верным, не слишком шаблонным и соответствовать задаче. Автоматическая метрика обычно видит только часть этих требований.

7.2 Likelihood и perplexity

Если есть reference $y_1, \dots, y_T$, можно считать negative log-likelihood:

$$\text{NLL} = - \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t}, x).$$

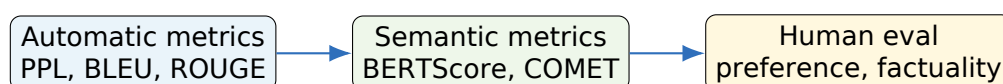
Perplexity:

$$\text{PPL} = \exp \left(\frac{1}{T} \text{NLL} \right).$$

Низкая perplexity означает, что модель хорошо предсказывает тестовый текст с точки зрения вероятности. Но это не гарантирует хороший пользовательский ответ: decoding strategy, factuality и безопасность могут быть важнее token-level likelihood.

7.3 Overlap-метрики

BLEU измеряет precision n-gram overlap между candidate и references, с brevity penalty за слишком короткий текст. ROUGE больше связан с recall n-grams и часто применяется в summarization. METEOR пытается учитывать формы слов и синонимы. Эти метрики дешевы и полезны для массовых экспериментов, но плохо видят парафразы.



Чем более открытая задача, тем меньше достаточно одной автоматической метрики.

Рис. 27: Оценка генерации обычно комбинирует несколько взглядов на качество.

7.4 Semantic metrics и human evaluation

BERTScore сравнивает contextual embeddings токенов candidate и reference, поэтому лучше переносит перефразировки. COMET и похожие метрики обучаются приближать человеческие оценки качества перевода. Но они сами являются моделями и могут наследовать bias.

Для open-ended generation часто требуется human evaluation. Людей просят оценить fluency, adequacy, relevance, factuality, coherence, harmlessness или выбрать лучший ответ из пары. В практических системах используют task-specific метрики: pass@k для кода, exact match/F1 для QA, factual consistency для summarization, task success rate для диалогов.

Формулы и определения

Хорошая evaluation-схема обычно содержит:

1. быструю automatic metric для итераций;
2. task-specific metric, связанную с реальной целью;
3. ручную или preference evaluation для качества, которое плохо формализуется;
4. qualitative error analysis на примерах.

Как отвечать на экзамене

Качество text generation измеряют через likelihood/perplexity, n-gram overlap метрики вроде BLEU и ROUGE, semantic metrics вроде BERTScore или COMET, а также human evaluation. Perplexity хороша для language modeling, но не всегда коррелирует с пользовательским качеством. BLEU/ROUGE дешевы, но плохо учитывают парафразы. Для открытой генерации нужны human/preference оценки и task-specific checks.

Частые ошибки

Не говорите, что BLEU полностью измеряет качество перевода. BLEU - приближенная automatic metric. Также нельзя сравнивать модели по automatic metric, если у них разные decoding strategies и это не контролировалось.

8. Unsupervised approach to machine translation

Интуиция

Supervised MT использует пары предложений “исходный текст - перевод”. Unsupervised MT пытается обойтись без таких пар: есть только тексты на каждом языке отдельно. Главная идея - построить мост между языками через shared representations и back-translation.

8.1 Что значит unsupervised MT

В supervised machine translation есть parallel corpus: пары (x, y) , где y является переводом x . В unsupervised MT параллельных пар нет. Доступны monolingual corpora на двух языках X и Y . Задача - научить модели $M_{X \rightarrow Y}$ и $M_{Y \rightarrow X}$ переводить между языками.

Метод опирается на несколько предположений. Во-первых, семантические структуры языков частично похожи: похожие смыслы образуют похожую геометрию. Во-вторых, модель можно инициализировать так, чтобы предложения разных языков попадали в общий latent space. В-третьих, хороший перевод должен быть не только смысловым соответствием source, но и естественным текстом target language.

8.2 Denoising autoencoding

Модель учится восстанавливать предложение из зашумленной версии на том же языке:

$$\tilde{x} = \text{noise}(x), \quad x \approx \text{Decoder}(\text{Encoder}(\tilde{x})).$$

Это учит encoder-decoder строить устойчивые представления и генерировать грамматичный текст на каждом языке.

8.3 Back-translation

Back-translation превращает monolingual data в synthetic parallel data. Пусть есть предложение y на языке Y . Текущая модель $M_{Y \rightarrow X}$ переводит его обратно в X , получаем псевдо-source $\hat{x}$. Тогда пара $(\hat{x}, y)$ используется как обучающий пример для $M_{X \rightarrow Y}$. Симметрично строятся пары для обратного направления.

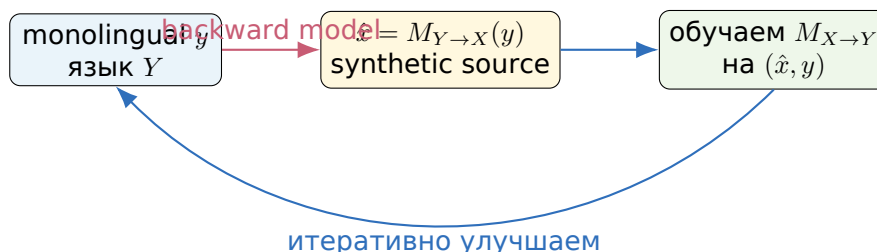


Рис. 28: Back-translation создает псевдопараллельные данные из monolingual corpora.

8.4 Инициализация и ограничения

Unsupervised MT трудно запустить с нуля. Часто нужны cross-lingual word embeddings, общий subword vocabulary, shared encoder/decoder, multilingual pretraining или seed dictionary. Подход лучше работает для близких языков, похожих доменов и больших monolingual corpora. Для далеких языков, разных письменностей и low-resource условий качество обычно хуже.

Как отвечать на экзамене

Unsupervised MT обучается без parallel corpus, используя monolingual corpora двух языков. Основные компоненты: общий latent space или shared vocabulary, denoising autoencoder для fluency, back-translation для создания synthetic parallel data и cycle consistency. Модели в двух направлениях попеременно переводят monolingual предложения, создают псевдопары и обучаются на них. Качество зависит от инициализации, близости языков и домена.

Частые ошибки

Unsupervised не означает “без данных”. Данные есть, просто они не параллельные. Самая частая ошибка - забыть back-translation, хотя именно она делает обучение перевода возможным.

9. Self-attention mechanism

Интуиция

В self-attention каждый токен сам решает, какие другие токены последовательности ему нужны для обновления представления. Это как attention, но query, keys и values строятся из одной и той же последовательности.

9.1 Формальная запись

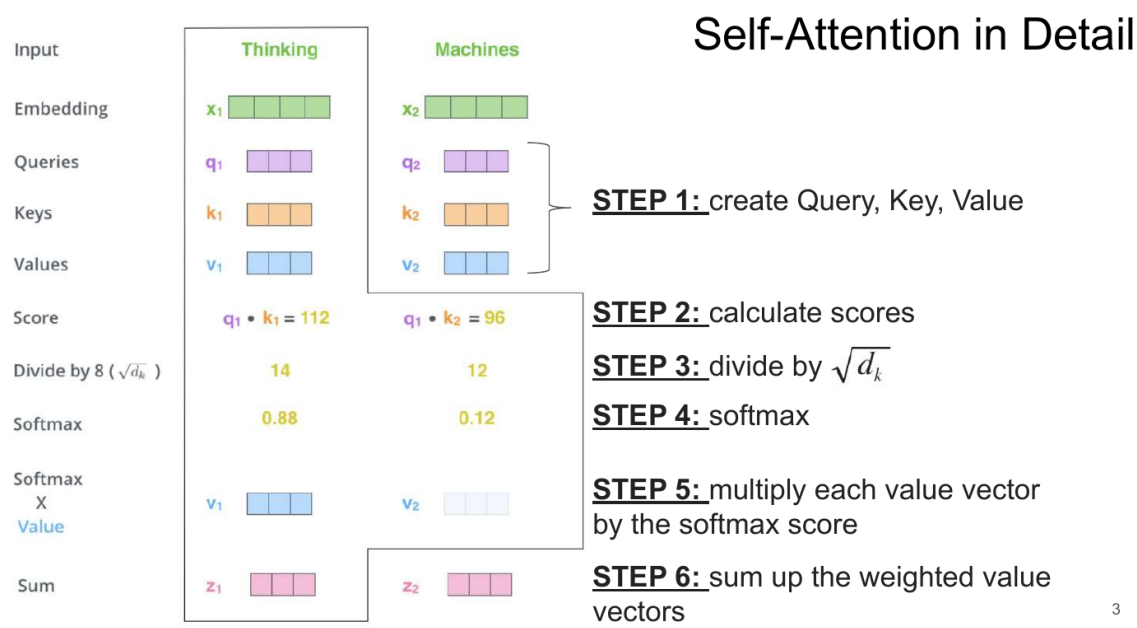


Рис. 29: Self-attention пошагово. Self-attention строится в шесть шагов: создать Q/K/V, посчитать scores, поделить на $\sqrt{d_k}$, применить softmax, взвесить values и сложить weighted value vectors.

Multi-Head Attention

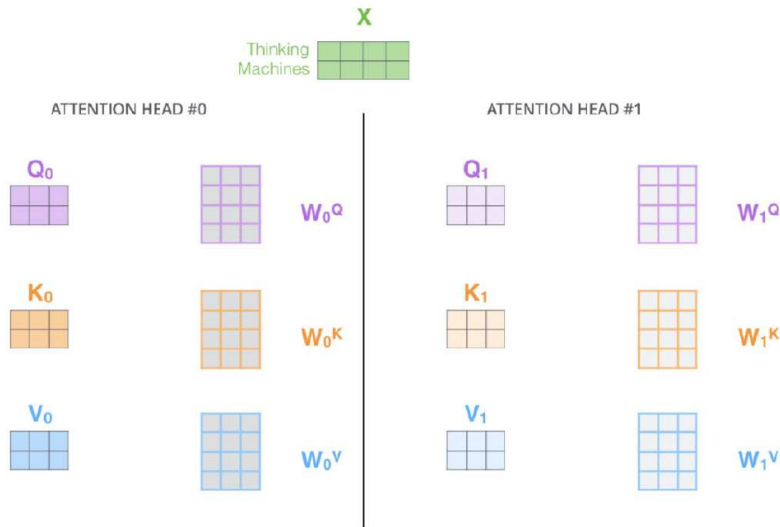


Рис. 30: Multi-head attention. Несколько attention heads позволяют параллельно смотреть на разные типы отношений между токенами, после чего результаты конкатенируются и проецируются обратно.

Пусть входные представления токенов образуют матрицу $X \in \mathbb{R}^{T \times d}$. Для каждого токена строятся query, key и value:

$$Q = XW_Q, K = XW_K, \quad V = XW_V.$$

Затем

$$\text{SelfAttention}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Матрица $A = \text{softmax}(QK^\top / \sqrt{d_k})$ имеет размер $T \times T$. Элемент A_{ij} показывает, насколько токен i смотрит на токен j .

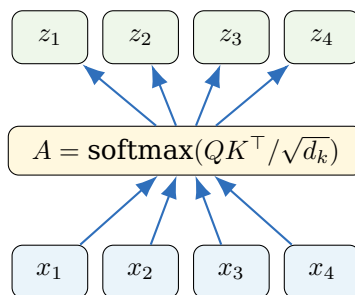


Рис. 31: Self-attention обновляет представление каждого токена через взвешенную сумму value-векторов всех токенов.

9.2 Маски

В encoder self-attention обычно можно смотреть на все непустые токены. Padding mask запрещает attention к padding positions. В decoder language model нужна causal mask: позиция t может смотреть только на позиции $\leq t$, иначе модель увидит будущие токены и задача обучения станет нечестной.

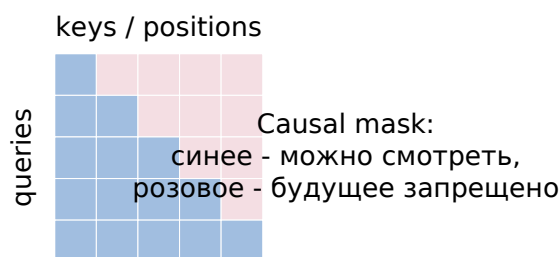


Рис. 32: Causal mask в decoder self-attention.

9.3 Multi-head attention

Одна head может выучить один тип отношений. Multi-head attention запускает несколько attention mechanisms параллельно:

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V),$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O.$$

Разные heads могут захватывать локальные связи, синтаксис, coreference, long-range dependencies. Это не гарантировано для каждой головы, но как архитектурная идея увеличивает выразительность.

9.4 Порядок и сложность

Self-attention сама по себе permutation-equivariant: если переставить токены, модель не знает, какой порядок был изначально. Поэтому нужны positional encodings или positional embeddings. Классический Transformer использует sinusoidal encodings, BERT - learned absolute positions, современные decoder-only модели часто используют rotary embeddings.

Цена self-attention - $O(T^2)$ по памяти и вычислениям, потому что строится матрица attention размера $T \times T$. Это главное ограничение для длинных контекстов.

Как отвечать на экзамене

Self-attention строит Q, K, V из одной последовательности и вычисляет $\text{softmax}(QK^\top / \sqrt{d_k})V$. Каждый token обновляется как weighted sum value-векторов других токенов. В encoder токены обычно смотрят на все позиции, в decoder используется causal mask. Multi-head attention дает несколько параллельных подпространств внимания. Поскольку self-attention не кодирует порядок сама по себе, нужны positional encodings. Сложность - $O(T^2)$.

Частые ошибки

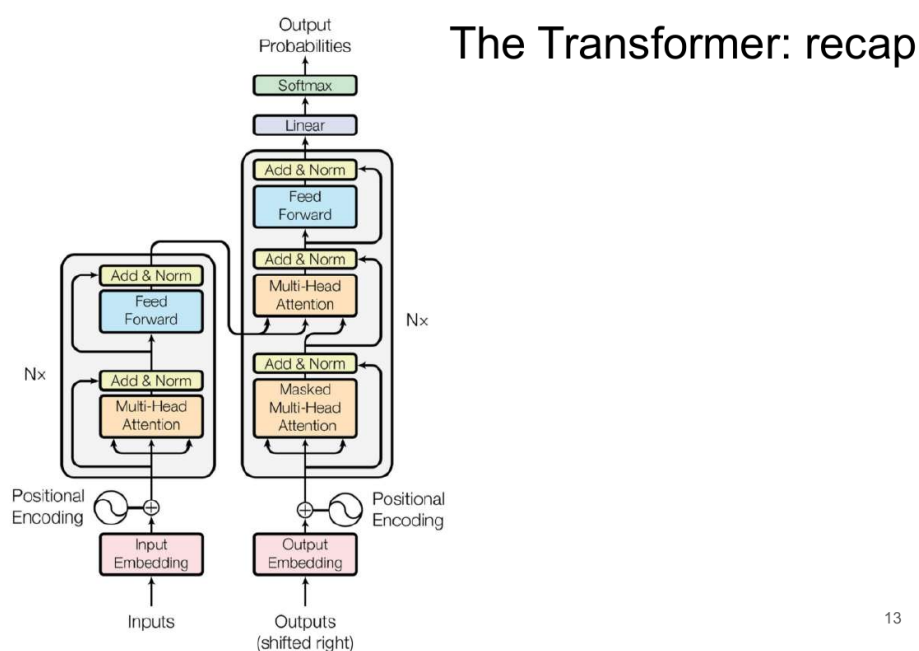
Типичные ошибки: забыть scaling by $\sqrt{d_k}$, забыть positional encoding, перепутать padding mask и causal mask, сказать, что self-attention всегда смотрит на все токены. В decoder она не должна смотреть в будущее.

10. Transformer architecture

Интуиция

Transformer заменяет рекуррентное чтение текста на набор блоков self-attention и feed-forward networks. Благодаря этому обучение хорошо параллелизуется, а токены могут напрямую обмениваться информацией.

10.1 Из чего состоит Transformer block



13

Рис. 33: Transformer recap. Transformer - это не только attention: в каждом блоке есть multi-head attention, feed-forward сеть, residual connections и normalization. Эти компоненты вместе делают глубокую архитектуру обучаемой.

Входной токен превращается в сумму token embedding и positional encoding:

$$e_t = \text{TokenEmbedding}(x_t) + \text{PositionalEncoding}(t).$$

Encoder block состоит из multi-head self-attention, residual connection, layer normalization, затем position-wise feed-forward network, еще residual и normalization. В post-norm записи:

$$Z = \text{LayerNorm}(X + \text{MHA}(X)),$$

$$Y = \text{LayerNorm}(Z + \text{FFN}(Z)).$$

Feed-forward network применяется независимо к каждой позиции:

$$\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2.$$

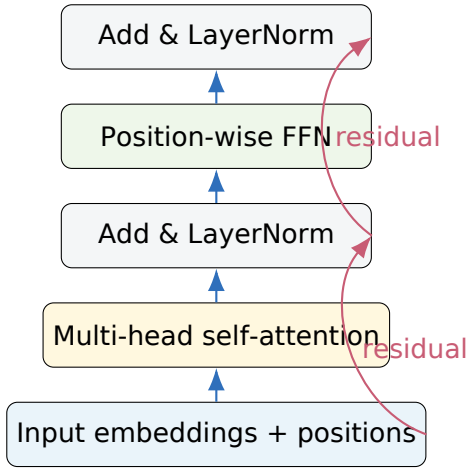


Рис. 34: Схема encoder-блока Transformer. В современных реализациях часто используется pre-norm, но смысл residual/normalization сохраняется.

10.2 Decoder block

Decoder block добавляет два отличия. Во-первых, masked self-attention, чтобы токен не видел будущее. Во-вторых, cross-attention к encoder outputs, если это encoder-decoder модель. В cross-attention queries приходят из decoder states, а keys и values - из encoder outputs:

$$Q = H_{dec}W_Q, \quad K = H_{enc}W_K, \quad V = H_{enc}W_V.$$

10.3 Зачем residual и LayerNorm

Residual connections позволяют градиентам проходить через глубокую сеть и дают слою возможность учить поправку к представлению, а не полностью новое представление. LayerNorm стабилизирует распределения активаций внутри одного примера и делает обучение глубоких последовательных моделей устойчивее.

10.4 Encoder-only, decoder-only, encoder-decoder

Encoder-only модели, например BERT, используют bidirectional self-attention и хороши для понимания текста: classification, NER, retrieval, sentence pair tasks. Decoder-only модели, например GPT-подобные, используют causal mask и хороши для генерации слева направо. Encoder-decoder модели, например T5/BART-подобные и классический Transformer для перевода, удобны для задач “входная последовательность → выходная последовательность”: translation, summarization, text-to-text.

Семейство	Что видит	Где удобно использовать
Encoder-only	Весь вход bidirectionally	классификация, NER, retrieval, extractive QA
Decoder-only	Только прошлые токены через causal mask	language modeling, диалог, open-ended generation
Encoder-decoder	Encoder видит source, decoder генерирует target	перевод, суммаризация, text-to-text задачи

Как отвечать на экзамене

Transformer состоит из attention-блоков и feed-forward networks с residual connections и LayerNorm. Encoder использует self-attention по всему входу, decoder - masked self-attention и, в encoder-decoder варианте, cross-attention к encoder outputs. Positional encodings добавляют информацию о порядке. Transformer хорошо параллелится, моделирует long-range dependencies и стал базой для BERT, GPT, T5 и мультимодальных моделей.

Частые ошибки

Не описывайте Transformer как “просто attention”. В блоке есть FFN, residual connections, normalization, positional information и разные виды attention. Также важно различать encoder self-attention, decoder masked self-attention и cross-attention.

11. BERT pretraining. MLM and NSP objectives

Интуиция

BERT - это Transformer encoder, который учится понимать текст bidirectionally. Но если просто попросить encoder предсказывать следующий токен, он будет видеть и левый, и правый контекст. Поэтому BERT использует masked language modeling.

11.1 Вход BERT

Типичный вход BERT для пары предложений:

[CLS] sentence A [SEP] sentence B [SEP].

К token embeddings добавляются position embeddings и segment embeddings. Segment embeddings показывают, к какому сегменту относится токен: А или В. Представление [CLS] часто используется как агрегированный вектор для классификационных задач.

11.2 Masked Language Modeling

BERT: pre-training

- “Masked Language Model” approach
- To make BERT better at handling relationships between multiple sentences, the pre-training process includes an additional task:
“Given two sentences (A and B), is B likely to be the sentence that follows A, or not?”

Рис. 35: BERT pre-training. BERT использует Masked Language Modeling и дополнительную задачу Next Sentence Prediction, чтобы encoder научился bidirectional contextual representations.

В MLM случайно выбираются позиции M , и модель должна восстановить исходные токены по контексту. В оригинальной схеме выбирается около 15% токенов. Из выбранных 80% заменяются на [MASK], 10% - на случайный токен, 10% остаются без изменения. Loss считается только на выбранных позициях:

$$\mathcal{L}_{MLM} = - \sum_{i \in M} \log p_{\theta}(x_i | x_{\setminus M}).$$

Схема 80/10/10 нужна, чтобы уменьшить mismatch между pretraining и fine-tuning: во время fine-tuning токена [MASK] обычно нет.

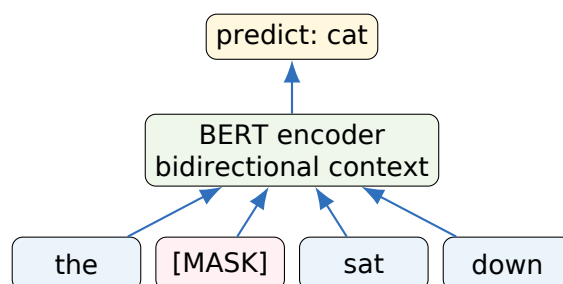


Рис. 36: MLM заставляет BERT восстанавливать скрытый токен по левому и правому контексту.

11.3 Next Sentence Prediction

NSP - бинарная задача для пары сегментов A и B. Нужно предсказать, является ли B настоящим следующим предложением после A. Метки: IsNext и NotNext. Классификация обычно строится по hidden state токена [CLS]:

$$p = \sigma(w^T h_{[CLS]} + b).$$

Общий loss оригинального BERT:

$$\mathcal{L}_{BERT} = \mathcal{L}_{MLM} + \mathcal{L}_{NSP}.$$

Полезность NSP в последующих работах обсуждалась, но для экзамена важно знать оригинальную постановку.

Как отвечать на экзамене

BERT - encoder-only Transformer, предобученный на MLM и NSP. В MLM выбирается часть токенов, обычно 15%; из них 80% заменяются на [MASK], 10% на случайный токен, 10% остаются без изменения; loss считается только на выбранных позициях. В NSP по паре предложений модель предсказывает, является ли второе продолжением первого. Pretraining дает bidirectional contextual representations, которые затем адаптируются к downstream tasks.

Частые ошибки

Не называйте BERT left-to-right language model. BERT не генерирует текст как causal decoder. Еще одна ошибка - считать MLM loss по всем токенам; в оригинальной постановке он считается только по выбранным позициям.

12. BERT fine-tuning. Transfer learning paradigm. Fine-tuning strategies

Интуиция

Pretraining учит общие языковые представления на огромном корпусе. Fine-tuning адаптирует эти представления к конкретной задаче, где размеченных данных может быть мало.

12.1 Transfer learning в NLP

BERT: fine-tuning for different tasks

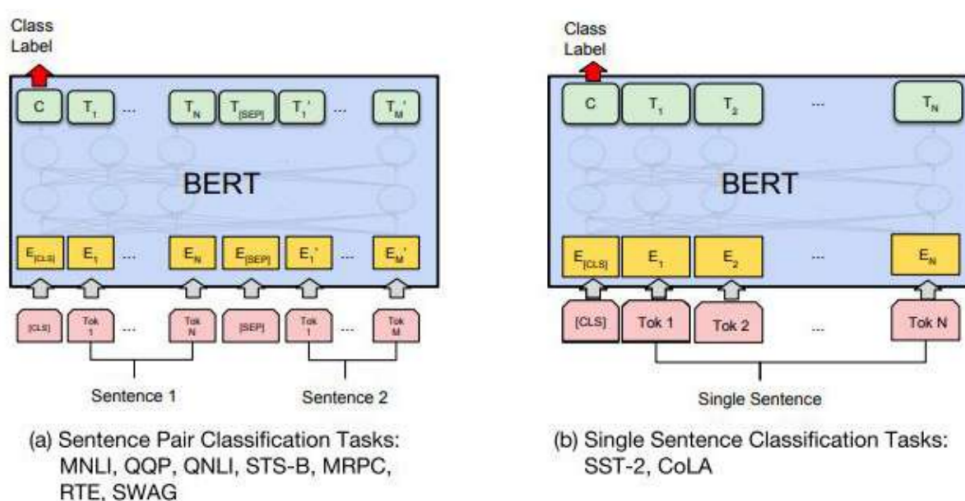


Рис. 37: BERT fine-tuning. Fine-tuning меняет только постановку входа и task-specific head: один и тот же pretrained encoder адаптируется к классификации, QA, NER и другим задачам.

Transfer learning состоит из двух фаз. На pretraining модель решает self-supervised задачи на больших неразмеченных данных. На fine-tuning добавляется маленькая task-specific head, и модель обучается на целевой задаче. Идея проста: языковые знания дороги, их выгодно выучить один раз и переносить.

12.2 Типовые heads

Для sentence classification берут final hidden state [CLS] и подают в linear classifier:

$$\hat{y} = \text{softmax}(Wh_{[CLS]} + b).$$

Для token classification, например NER, классифицируется hidden state каждого токена. Для extractive QA предсказываются start и end позиции answer span:

$$p_{start} = \text{softmax}(W_s H), \quad p_{end} = \text{softmax}(W_e H).$$

Для sentence pair tasks вход имеет вид [CLS] A [SEP] B [SEP], а решение часто принимается по [CLS].

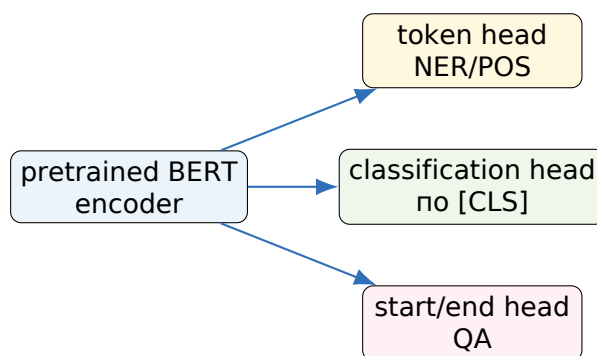


Рис. 38: Один pretrained encoder можно адаптировать к разным downstream tasks через разные heads.

12.3 Стратегии fine-tuning

Самый простой вариант - full fine-tuning: обновляются все параметры BERT и task head. Обычно нужен маленький learning rate, потому что pretrained weights легко разрушить. Этот вариант часто дает лучшее качество, но требует больше памяти и может переобучаться на маленьких данных.

Frozen encoder или feature extraction: BERT заморожен, обучается только head. Это дешевле и устойчивее на очень малых данных, но хуже адаптируется к домену. Между этими крайностями есть gradual unfreezing, discriminative learning rates, layer-wise learning rate decay, adapters, LoRA и prompt/prefix tuning. Все они пытаются сохранить полезные pretrained representations и при этом позволить модели подстроиться под задачу.

Полезное замечание

Для retrieval не всегда хорошо брать сырой [CLS] из BERT как sentence embedding. Часто нужны специальные sentence-transformer или contrastive fine-tuning методы, потому что обычный BERT не обязан делать косинусную геометрию предложений удобной для поиска.

Как отвечать на экзамене

BERT fine-tuning - это transfer learning: берем pretrained BERT, добавляем task-specific head и обучаем на целевой задаче. Для classification часто используют [CLS], для token tasks - hidden states токенов, для extractive QA - start/end heads. Стратегии включают full fine-tuning, frozen encoder, gradual unfreezing, разные learning rates по слоям, adapters, LoRA и prompt tuning. Главное - не обучать модель с нуля, а адаптировать pretrained representations.

Частые ошибки

Не используйте слишком большой learning rate в объяснении fine-tuning: это может разрушить pretrained модель. Не говорите, что fine-tuning всегда требует обновлять все параметры; существуют parameter-efficient стратегии.

13. Contrastive learning paradigm. Examples

Интуиция

В contrastive learning модель учится не предсказывать класс, а строить пространство сходства: позитивные пары должны быть близко, негативные - далеко.

13.1 Позитивы и негативы



A formulation of contrastive learning

What we want:

$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$$

x : reference sample; x^+ positive sample; x^- negative sample

Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .



53

Рис. 39: Формулировка contrastive learning. Для anchor x есть positive x^+ и negatives x^- . Encoder должен давать высокий score позитивной паре и низкие scores негативным.

A formulation of contrastive learning

Loss function given 1 positive sample and $N - 1$ negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

Commonly known as the InfoNCE loss ([van den Oord et al., 2018](#))

A lower bound on the mutual information between $f(x)$ and $f(x^+)$

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

The larger the negative sample size (N), the tighter the bound

Detailed derivation: [Poole et al., 2019](#)

58

Рис. 40: InfoNCE. InfoNCE можно понимать как cross-entropy задачу: среди N кандидатов нужно найти один positive. Чем больше негативов, тем сильнее контрастный обучающий сигнал.

SimCLR: A Simple Framework for Contrastive Learning

Cosine similarity as the score function:

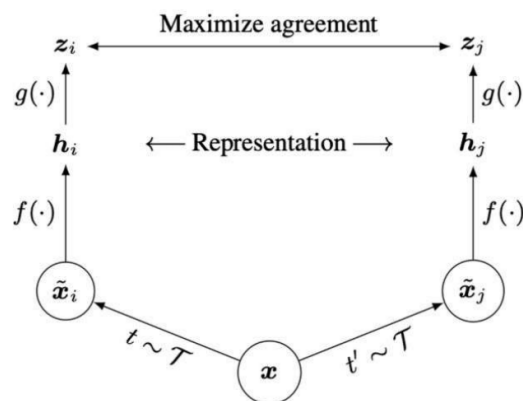
$$s(u, v) = \frac{u^T v}{\|u\| \|v\|}$$

Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied

Generate positive samples through data augmentation:

- random cropping, random color distortion, and random blur.

Source: [Chen et al., 2020](#)



59

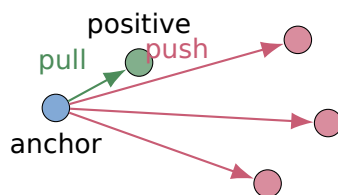
Рис. 41: SimCLR. SimCLR получает positive samples через аугментации одного изображения и применяет contrastive loss в projection space. Идея переносится на другие модальности, включая тексты и image-text пары.

Позитивная пара - два объекта, которые должны иметь близкие embeddings: две аугментации одного изображения, два перефразированных предложения, изображение и его caption. Негативная пара - объекты, которые считаются несоответствующими.

Пусть $z_i = f_\theta(x_i)$ - embedding. Для anchor i есть positive j и набор кандидатов. Одна популярная loss - InfoNCE:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)}.$$

Здесь τ - temperature. Малое τ делает softmax более резким, заставляя positive быть существенно ближе negatives.



Contrastive loss формирует геометрию embeddings.

Рис. 42: Позитивы притягиваются, негативы отталкиваются.

13.2 Почему нужны negatives и что такое collapse

Если только сближать позитивные пары, модель может сделать все embeddings одинаковыми: тогда расстояние между позитивами минимально, но представление бесполезно. Негативы или специальные anti-collapse механизмы не дают такому решению стать оптимальным.

In-batch negatives делают обучение эффективным: если в batch есть пары (x_i, y_i) , то для x_i правильный positive - y_i , а все остальные y_j считаются negatives. Это особенно важно для CLIP и sentence embeddings.

13.3 Примеры

SimCLR берет изображение, делает две аугментации и считает их positive pair; остальные изображения в batch - negatives. MoCo использует momentum encoder и queue, чтобы иметь много negatives без огромного batch. Sentence-BERT учит embeddings предложений для semantic search. Supervised contrastive learning считает позитивами объекты одного класса. Word2vec negative sampling тоже близок по духу: реальные пары слово-контекст противопоставляются случайным.

Как отвечать на экзамене

Contrastive learning обучает representation space через сравнение объектов. Позитивные пары сближаются, негативные отдаляются. Типичная loss - InfoNCE: positive должен иметь наибольшую similarity среди кандидатов, softmax делится на temperature. Примеры: SimCLR, MoCo, Sentence-BERT, supervised contrastive learning, CLIP. Цель - получить embeddings, пригодные для retrieval, clustering, zero-shot и transfer learning.

Частые ошибки

Не сводите contrastive learning к обычной классификации. Классификация учит границы между фиксированными классами, contrastive learning учит пространство сходства, которое можно переносить на новые задачи.

14. Как CLIP обучали для совместных embeddings текста и изображений

Интуиция

CLIP учит две модели - image encoder и text encoder - помещать соответствующие изображения и подписи рядом в одном embedding space. После этого текстовая фраза может играть роль “класса”.

14.1 Данные и архитектура

Обучающие данные CLIP - пары image-text из интернета: изображение и связанное с ним текстовое описание. Архитектурно есть два encoder'а:

- image encoder, например ResNet или Vision Transformer;
- text encoder, Transformer для токенизированного текста.

Оба encoder'а выдают embeddings одинаковой размерности. Обычно embeddings нормируются, а similarity считается через cosine similarity или dot product нормированных векторов.

14.2 Batch contrastive objective

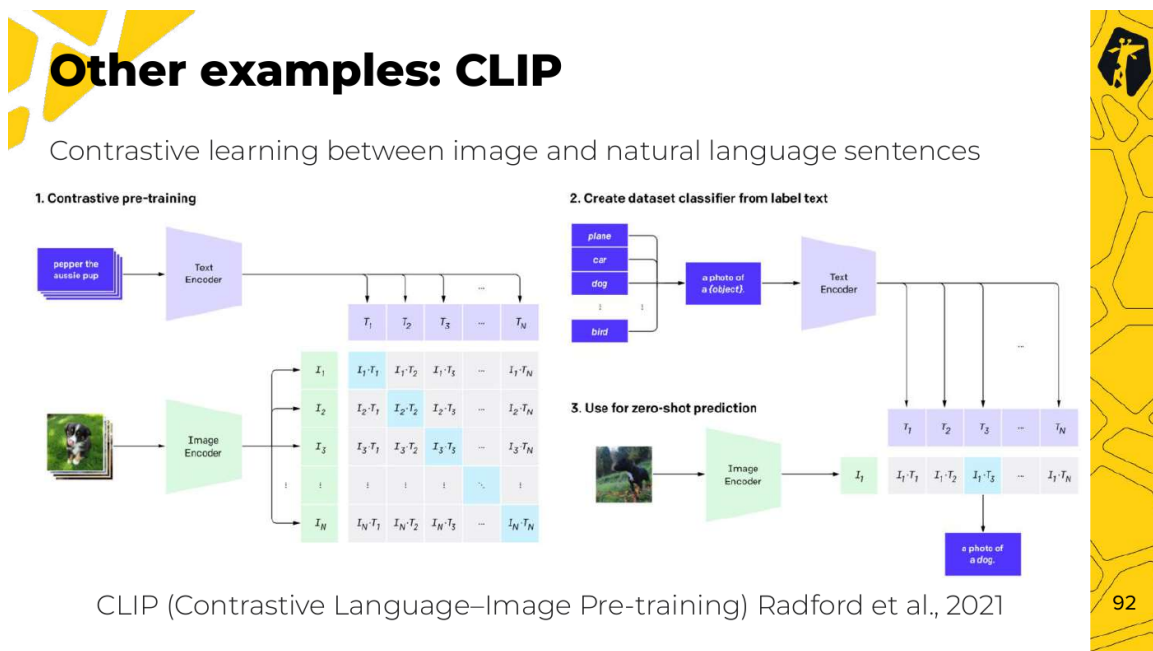


Рис. 43: CLIP. CLIP - пример contrastive learning между изображением и естественно-языковым описанием: image encoder и text encoder учатся попадать в общее embedding space.

Пусть в batch N пар (I_i, T_i) . Считаем матрицу similarities

$$S_{ij} = \frac{\langle f_I(I_i), f_T(T_j) \rangle}{\tau}.$$

Правильные пары находятся на диагонали. Loss симметричный: image-to-text классифицирует правильный текст для изображения, text-to-image классифицирует правильное изображение для текста:

$$\mathcal{L} = \frac{1}{2} \left(\text{CE}(S, \text{diag}) + \text{CE}(S^T, \text{diag}) \right).$$

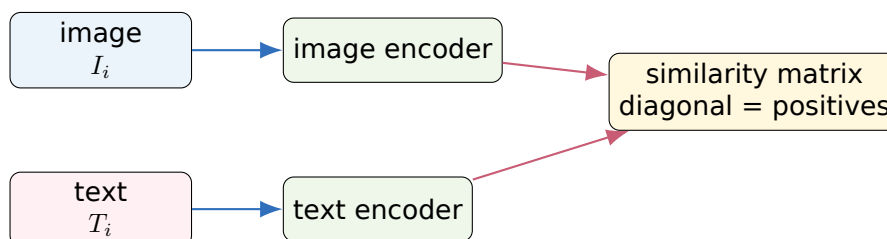


Рис. 44: CLIP обучает два encoder'а через контрастивную матрицу similarities в batch.

14.3 Почему получается zero-shot classification

Для классификации изображения можно написать текстовые prompts: “a photo of a cat”, “a photo of a dog”, “a photo of a car”. Text encoder превращает их в class embeddings. Image encoder превращает изображение в image embedding. Предсказывается класс с максимальной similarity. Таким образом новые классы задаются текстом, без обучения классификатора на этих классах.

14.4 Ограничения

CLIP хорошо использует масштаб и естественную связь картинок с текстом, но не является идеальным визуальным рассуждателем. Он может быть чувствителен к prompt wording, bias в данных, тексту на изображении, редким классам и композиционным запросам. Для экзамена главное - объяснить contrastive batch objective и zero-shot механизм через текстовые prompts.

Как отвечать на экзамене

CLIP обучали на парах image-text. Image encoder и text encoder выдают embeddings в общее пространство. В batch строится матрица similarities между всеми изображениями и всеми текстами; диагональные пары являются позитивами, остальные элементы batch - in-batch negatives. Loss - симметричная contrastive cross-entropy image-to-text и text-to-image. Zero-shot classification получается потому, что классы можно представить текстовыми prompts и выбрать prompt с максимальной similarity к image embedding.

Частые ошибки

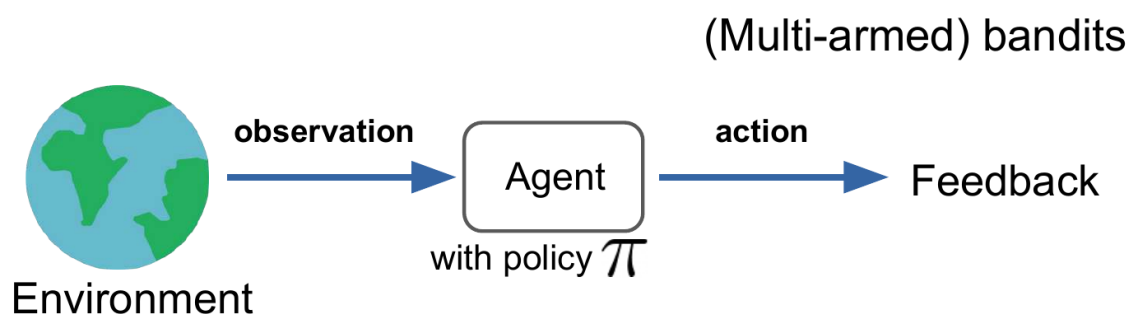
Не говорите, что CLIP обучался как обычный классификатор ImageNet-классов. Его ключевая идея - contrastive alignment изображений и текстов в общем embedding space.

15. Reinforcement Learning: постановка задачи и отличие от Supervised Learning

Интуиция

В supervised learning правильный ответ дан в датасете. В reinforcement learning агент сам действует в среде, получает награды и должен понять, какие действия привели к хорошему будущему результату.

15.1 Основные сущности RL



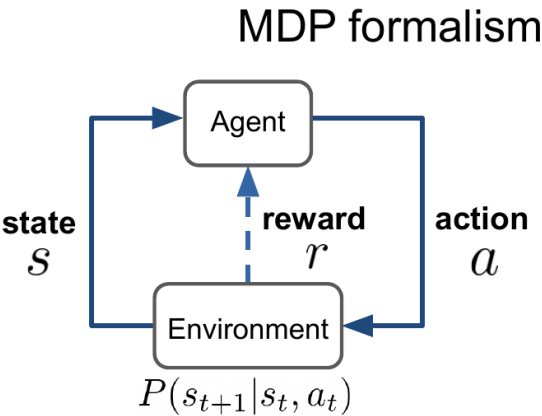
- Observation (state): vector or image or sequence ... or *nothing*
- Policy: mapping from state to action
- Action
- Feedback (reward): usually a converted to a number

9

Рис. 45: Agent-environment loop. RL описывает замкнутый цикл: агент получает observation/state, выбирает action по policy, среда возвращает reward и новое состояние. Данные зависят от действий самого агента.

- State: $s \in \mathcal{S}$
- Action: $a \in \mathcal{A}$
- Reward: $r \in \mathbb{R}$

- Dynamics: $P(s_{t+1}|s_t, a_t)$



Markov property:

$$P(s_{t+1}|s_t, a_t, \dots, s_0, t_0) = P(s_{t+1}|s_t, a_t)$$

Рис. 46: MDP formalism. MDP задает state, action, reward и dynamics. Markov property означает, что будущее условно зависит только от текущего state и action, а не от всей истории.

RL описывает взаимодействие агента и среды. В момент времени t агент наблюдает состояние s_t , выбирает действие a_t по policy $\pi(a|s)$, среда возвращает награду r_t и новое состояние s_{t+1} . Цель - максимизировать ожидаемую суммарную награду.

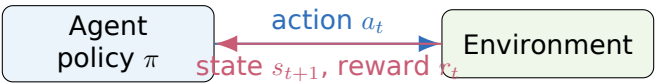


Рис. 47: Цикл взаимодействия агента со средой.

Policy может быть deterministic: $a = \pi(s)$, или stochastic: $\pi(a|s) = P(A_t = a|S_t = s)$. Во многих задачах стохастическая policy удобна для exploration и для вывода policy gradient.

15.2 Чем RL отличается от supervised learning

В supervised learning данные обычно фиксированы, объекты считаются i.i.d., а для каждого объекта есть target. В RL данные зависят от текущей policy: поменяли агента - поменялась траектория. Награда может быть отложенной: действие сейчас повлияет на результат через много шагов. Кроме того, агент должен балансировать exploration и exploitation: пробовать новое или использовать уже известные хорошие действия.

Свойство	Supervised learning	Reinforcement learning
Данные	фиксированный датасет	зависят от policy
Цель	приблизить target	максимизировать reward
Сигнал качества	сразу для объекта	может быть delayed
Действия модели	не меняют датасет	меняют будущие состояния

Ключевая проблема

generalization

exploration + credit
assignment**Формулы и определения**

Типичная цель RL:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

где $\tau = (s_0, a_0, r_0, \dots)$ - trajectory, γ - discount factor.**Как отвечать на экзамене**

В RL агент взаимодействует со средой: наблюдает состояние, выбирает действие, получает награду и новое состояние. Цель - найти policy, максимизирующую ожидаемую суммарную discounted reward. В отличие от supervised learning, правильных действий обычно нет в датасете, награда может быть отложенной, данные зависят от policy, а агент должен исследовать среду.

Частые ошибки

Не объясняйте RL как “классификацию действий”. В некоторых задачах можно имитировать эксперта, но классический RL оптимизирует награду через взаимодействие со средой, а не просто повторяет labels.

16. Markov property

Интуиция

Состояние является марковским, если в нем уже содержится вся информация из прошлого, полезная для предсказания будущего. Тогда история не нужна: достаточно текущего состояния.

16.1 Определение

Процесс обладает Markov property, если

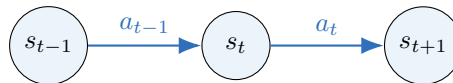
$$P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = P(S_{t+1} | S_t, A_t).$$

То есть будущее условно независимо от прошлого при известном текущем состоянии и действии.

16.2 Markov Decision Process

MDP обычно задается пятеркой $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, где $\mathcal{S}$ - состояния, $\mathcal{A}$ - действия, $P(s' | s, a)$ - transition probabilities, $R(s, a)$ или $R(s, a, s')$ - reward function, γ - discount factor.

Если состояние не содержит всей информации, возникает partially observable setting. Тогда агент видит observation o_t , но истинное state s_t может быть скрыто. Например, по одному кадру игры нельзя понять скорость объекта; нужно несколько кадров или recurrent memory.



Если s_t марковское, то для распределения s_{t+1} прошлые состояния уже не нужны.

Рис. 48: Марковское состояние “экранирует” прошлое от будущего.

16.3 Почему это важно

Уравнения Беллмана, value functions и большинство базовых RL-алгоритмов предполагают марковскую постановку. Если Markov property нарушается, одна и та же observation может требовать разных действий в зависимости от прошлого. Тогда нужна память: RNN-policy, belief state, stacking frames или более богатое state representation.

Как отвечать на экзамене

Markov property означает, что условное распределение будущего состояния при текущем состоянии и действии не зависит от всей предыдущей истории. MDP задается состояниями, действиями, transition probabilities, rewards и discount factor. Свойство важно, потому что Bellman equations и value-based методы используют текущий state как достаточную статистику прошлого. Если агент видит только неполное observation, задача становится partially observable.

Частые ошибки

Не путайте state и observation. Observation - то, что видит агент; state - полное описание, достаточное для марковского будущего. В простых задачах они совпадают, но не всегда.

17. Reward discounting. Cumulative reward

Интуиция

Агенту важна не только немедленная награда, но и будущие последствия действий. Discount factor задает, насколько сильно мы ценим будущие награды относительно текущих.

17.1 Return

Reward discounting

Рис. 49: Reward discounting. Discounting вводит коэффициент γ , который уменьшает вклад будущих наград. Это делает бесконечные суммы конечными, если награды ограничены и $\gamma < 1$.

Reward discounting

Get rid of infinite sum by **discounting** $0 \leq \gamma < 1$

$$G_t \triangleq R_t + \underset{\text{discount factor} \rightarrow}{\gamma} R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The same cake compared to today's one worth

- γ times less tomorrow
- γ^2 times less the day after tomorrow



γ will eat it day by day

Рис. 50: Интуиция discounting. Один и тот же reward в будущем считается менее ценным: завтра он стоит в γ раз меньше, послезавтра - в γ^2 раз меньше.

Return с момента t :

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Если эпизод конечный, сумма идет до конца episode. Если задача continuing, discount $\gamma < 1$ помогает сделать сумму конечной и уменьшает влияние далекого будущего.

Рекурсивная форма:

$$G_t = r_t + \gamma G_{t+1}.$$

Эта простая формула лежит в основе Bellman equations.

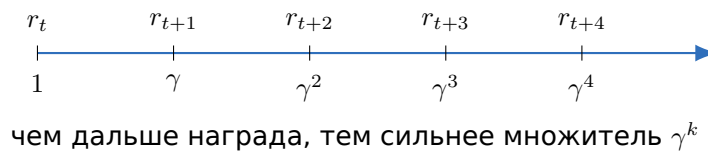


Рис. 51: Discounted return как взвешенная сумма будущих наград.

17.2 Зачем нужен discounting

Discounting имеет несколько смыслов. Во-первых, математический: для бесконечных процессов сумма может сходиться. Во-вторых, практический: дальнейшее будущее менее надежно предсказывается. В-третьих, поведенческий: можно задавать степень “терпеливости” агента. Если γ близко к 0, агент почти жадный и думает о ближайшей награде. Если γ близко к 1, он учитывает долгосрочные последствия.

17.3 Пример

Пусть rewards равны 1, 2, 3, $\gamma = 0.9$. Тогда

$$G_0 = 1 + 0.9 \cdot 2 + 0.9^2 \cdot 3 = 1 + 1.8 + 2.43 = 5.23.$$

Если $\gamma = 0.1$, то $G_0 = 1 + 0.2 + 0.03 = 1.23$, и будущие награды почти не важны.

Как отвечать на экзамене

Cumulative reward или return - это сумма будущих наград, обычно с discount factor: $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$. Discount $\gamma \in [0, 1]$ задает важность будущего, помогает в continuing tasks и дает рекурсивную формулу $G_t = r_t + \gamma G_{t+1}$. При маленьком γ агент ориентируется на немедленные награды, при γ близком к 1 - на долгосрочные.

Частые ошибки

Не говорите, что discount всегда “вероятность дожить до будущего”. Иногда это возможная интерпретация, но в RL чаще это гиперпараметр, определяющий горизонт планирования и сходимость return.

18. Value function. Как вычислить для session?

Интуиция

Value function отвечает на вопрос: “насколько хорошо оказаться в этом состоянии, если дальше следовать policy?” Для одной записанной session value можно оценить фактическим return из этого состояния.

18.1 Определение

Для policy π state value:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s].$$

Action value:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a].$$

В этом разделе фокус на V . Она усредняет будущую discounted reward по траекториям, которые начинаются из s и дальше следуют π .

18.2 Вычисление для одной session

Пусть есть episode:

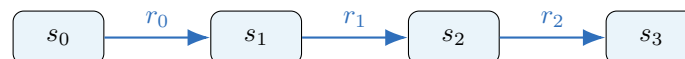
$$s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T.$$

Для каждого момента можно посчитать empirical return назад:

$$G_T = 0 \quad \text{или terminal value,}$$

$$G_t = r_t + \gamma G_{t+1}.$$

Тогда наблюдение G_t является sample estimate для $V^\pi(s_t)$. Если состояние встречается несколько раз или есть много episodes, оценки усредняются.



$$G_1 = r_1 + \gamma r_2 + \dots$$

sample для $V(s_1)$

Рис. 52: Для одного episode value состояния оценивается return от момента посещения до конца.

18.3 Monte Carlo и TD

Monte Carlo ждет конца episode и использует полный return G_t . Это дает естественную оценку, но может иметь большую дисперсию и не подходит, если episode очень длинные или бесконечные.

Temporal Difference обновляет value, не дожидаясь конца:

$$V(s_t) \leftarrow V(s_t) + \alpha (r_t + \gamma V(s_{t+1}) - V(s_t)).$$

Величина в скобках - TD error. TD использует bootstrap: часть target берется из текущей оценки $V(s_{t+1})$.

Как отвечать на экзамене

Value function $V^\pi(s)$ - ожидаемый return из состояния s при следовании policy π . Для одной session можно посчитать returns backward: $G_t = r_t + \gamma G_{t+1}$, и использовать G_t как sample estimate для $V(s_t)$. При нескольких посещениях или episodes оценки усредняют. Monte Carlo использует полный return после конца episode, TD обновляет value по target $r_t + \gamma V(s_{t+1})$ без ожидания конца.

Частые ошибки

Не путайте фактический G_t в одной траектории и истинное $V^\pi(s)$: value - математическое ожидание по возможным будущим траекториям, а G_t - один sample этого ожидания.

19. Q-function. Как научиться ее оценивать двумя способами?

Интуиция

Value говорит, насколько хорошее состояние. Q-function говорит, насколько хорошо в этом состоянии выбрать конкретное действие. Поэтому по $Q(s, a)$ можно выбирать действия напрямую.

19.1 Определение

Action-value function:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Если известна оптимальная $Q^*(s, a)$, оптимальная policy может выбирать

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Связь с value:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)} Q^\pi(s, a).$$

19.2 Способ 1: Monte Carlo estimation

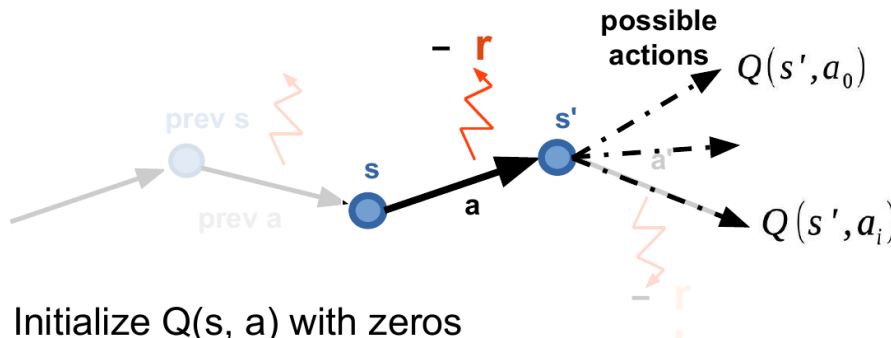
Можно собирать episodes, находить все посещения пары (s, a) и усреднять returns после этих посещений:

$$Q(s, a) \approx \frac{1}{N(s, a)} \sum_{i: S_i=s, A_i=a} G_i.$$

Это просто и не требует модели среды, но нужно дождаться конца episode. Дисперсия может быть большой.

19.3 Способ 2: Bellman / TD learning

Q-learning



Initialize $Q(s, a)$ with zeros

- Sample $\langle s, a, r, s' \rangle$ from the environment
- Compute new $Q(s, a)$ estimation:

$$\hat{Q}(s, a) = r(s, a) + \gamma \max_{a_i} Q(s', a_i)$$

- Update $Q(s, a)$:

$$Q(s, a) \leftarrow \alpha \cdot \hat{Q}(s, a) + (1 - \alpha) Q(s, a)$$

29

Рис. 53: Q-learning. Q-learning обновляет оценку $Q(s, a)$ по одному переходу $\langle s, a, r, s' \rangle$ и bootstrap-цели $r + \gamma \max_{a'} Q(s', a')$.

Для фиксированной policy SARSA использует on-policy target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha (r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)).$$

Q-learning использует off-policy target с максимумом:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right).$$

В deep RL таблицу Q заменяют нейросетью $Q_\theta(s, a)$, а target часто считают через frozen target network:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a').$$

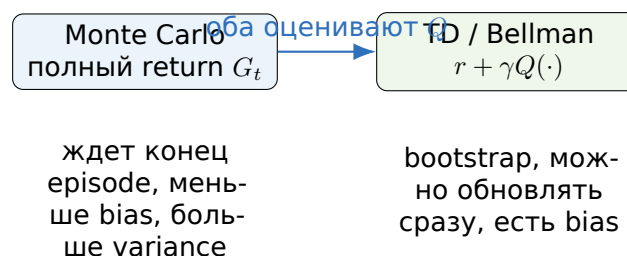


Рис. 54: Два базовых способа учить Q-function: по полному return или через Bellman bootstrap.

Как отвечать на экзамене

$Q^\pi(s, a)$ - ожидаемый return, если в состоянии s выбрать действие a , а дальше следовать policy π . Оценивать Q можно двумя базовыми способами. Monte Carlo усредняет фактические returns после посещения пары (s, a) . TD/Bellman методы

обновляют Q по bootstrap-target: SARSA использует $r + \gamma Q(s', a')$, Q-learning - $r + \gamma \max_{a'} Q(s', a')$. В deep Q-learning Q аппроксимируется нейросетью.

Частые ошибки

Не путайте SARSA и Q-learning. SARSA on-policy: следующий action берется из текущего поведения. Q-learning off-policy: target использует greedy максимум по действиям.

20. Как policy gradient оценивает градиент reward по параметрам policy

Интуиция

Если policy является нейросетью, хочется улучшать ее градиентным методом. Но reward приходит из среды и часто недифференцируем по параметрам policy. Policy gradient использует likelihood ratio trick: дифференцирует вероятность выбранных действий.

20.1 Objective

Пусть траектория $\tau = (s_0, a_0, r_0, \dots)$ порождается policy π_θ . Цель:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)],$$

где $R(\tau)$ - return траектории.

Распределение траектории:

$$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t).$$

Среда P не зависит от θ , но зависит вероятность действий.

20.2 Likelihood ratio trick

Logderivative trick

Simple math

$$\nabla \log \pi(z) = ???$$

(try chain rule)

Рис. 55: Log-derivative trick. Ключевой трюк: $\nabla \log \pi(z) = \nabla \pi(z) / \pi(z)$, значит $\nabla \pi(z) = \pi(z) \nabla \log \pi(z)$. Он превращает производную распределения в expectation.

REINFORCE (discounted)

- Policy gradient

$$\nabla J = E_{\substack{s \sim p(s) \\ a \sim \pi_\theta(s|a)}} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

- Approximate with sampling

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s, a \in \mathcal{Z}_i} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$$

40

Рис. 56: REINFORCE. Для discounted case в формуле policy gradient вместо immediate reward используется action value $Q(s, a)$, который на практике оценивают sampled return G_t .

Используем тождество

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau).$$

Тогда

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p_\theta(\tau)].$$

Так как от θ зависят только policy terms,

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

Получаем REINFORCE-estimator:

$$\nabla_\theta J(\theta) \approx \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t.$$

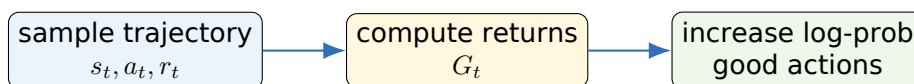


Рис. 57: Policy gradient повышает вероятность действий, после которых получился большой return.

20.3 Интуиция

Если после действия получился большой return, градиент увеличивает $\log \pi_\theta(a_t | s_t)$, то есть делает это действие более вероятным в похожем состоянии. Если return маленький или после вычитания baseline отрицательный, вероятность действия уменьшается. Метод не требует дифференцировать среду, но имеет высокую дисперсию, потому что оценивает ожидание по sampled trajectories.

Как отвечать на экзамене

Policy gradient оптимизирует $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} R(\tau)$. Через likelihood ratio trick градиент записывается как ожидание $R(\tau) \nabla_\theta \log p_\theta(\tau)$. Так как от параметров зависят только вероятности действий, получаем estimator $\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$. Интуитивно: действия, приведшие к высокому return, становятся более вероятными; действия с плохим return - менее вероятными.

Частые ошибки

Не надо говорить, что мы дифференцируем reward напрямую по action. В model-free policy gradient среда может быть недифференцируема; мы дифференцируем log-probability действия под policy.

21. Что такое baseline в policy gradient?

Интуиция

Policy gradient без baseline шумный: один и тот же хороший action может получить разную оценку из-за случайности будущего. Baseline вычитает “ожидаемый уровень” награды, оставляя advantage: насколько действие лучше обычного для этого состояния.

21.1 Формула с baseline

В REINFORCE можно заменить G_t на $G_t - b(s_t)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)) \right].$$

Если baseline не зависит от действия a_t , он не меняет математическое ожидание градиента, но может сильно уменьшить дисперсию.

21.2 Почему baseline не вносит bias

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) Q(s, a) - \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_{\theta}(a|s)}} \nabla \log \pi_{\theta}(a|s) b(s) = \dots$$

49

Рис. 58: Baseline в REINFORCE. Baseline $b(s)$ вычитается из $Q(s, a)$, чтобы уменьшить дисперсию градиентной оценки, но он не должен зависеть от action.

REINFORCE baselines

We can subtract arbitrary baseline $b(s)$

$$\nabla J = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(a|s)}} \nabla \log \pi_\theta(a|s) (Q(s, a) - b(s)) = \dots$$

$$\dots = \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(a|s)}} \nabla \log \pi_\theta(a|s) Q(s, a) - \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(a|s)}} \nabla \log \pi_\theta(a|s) b(s) = \dots$$

$$\mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(a|s)}} \nabla \log \pi_\theta(a|s) b(s) = b(s) \cdot \mathbb{E}_{\substack{s \sim p(s) \\ a \sim \pi_\theta(a|s)}} \nabla \log \pi_\theta(a|s) = 0$$

51

Рис. 59: Baseline не меняет направление градиента. Слайд показывает главный шаг доказательства: математическое ожидание $\nabla \log \pi(a|s)b(s)$ по действиям равно нулю, если b не зависит от a .

Для фиксированного состояния:

$$\mathbb{E}_{a \sim \pi} [\nabla_\theta \log \pi_\theta(a|s) b(s)] = b(s) \sum_a \pi(a|s) \nabla_\theta \log \pi(a|s).$$

Так как $\pi \nabla \log \pi = \nabla \pi$,

$$b(s) \sum_a \nabla_\theta \pi(a|s) = b(s) \nabla_\theta \sum_a \pi(a|s) = b(s) \nabla_\theta 1 = 0.$$

Поэтому baseline не сдвигает expected gradient.

21.3 Value baseline и advantage

Самый важный baseline - value function:

$$b(s) = V^\pi(s).$$

Тогда

$$G_t - V^\pi(s_t)$$

оценивает advantage: насколько фактический исход лучше ожидаемого из этого состояния. В actor-critic actor обновляет policy, critic оценивает value или Q и предоставляет baseline/advantage.

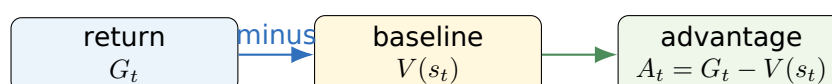


Рис. 60: Baseline превращает абсолютный return в относительную оценку качества действия.

Как отвечать на экзамене

Baseline в policy gradient - величина, которую вычитают из return, чтобы уменьшить дисперсию оценки градиента. Если baseline зависит только от состояния, но не от action, он не меняет expected gradient: ожидание $\nabla \log \pi(a|s)b(s)$ равно нулю. Частый baseline - value function $V(s)$. Тогда используется advantage $A(s, a) = Q(s, a) - V(s)$ или оценка $G_t - V(s_t)$. В actor-critic critic оценивает baseline, actor обновляет policy.

Частые ошибки

Baseline не является “штрафом” в reward. Он не меняет задачу оптимизации в математическом ожидании, а уменьшает variance gradient estimator. Важно: baseline не должен зависеть от выбранного action, иначе unbiasedness может нарушиться.

22. Почему прямое обучение через *likelihood* / *cross-entropy* не всегда достаточно для *language modeling*

Интуиция

Cross-entropy учит модель предсказывать следующий токен в датасете. Это мощная и базовая *objective*. Но пользователь оценивает не каждый токен отдельно, а весь ответ: полезность, фактичность, стиль, безопасность и достижение цели.

22.1 Что оптимизирует *cross-entropy*

Для *language modeling* или *seq2seq* с *teacher forcing* оптимизируется

$$\mathcal{L}_{CE} = - \sum_{t=1}^T \log p_{\theta}(y_t | y_{<t}, x).$$

Это *maximum likelihood estimation* для *training data*. Она делает модель хорошим предсказателем токенов из распределения данных. Это необходимая база: без *CE* современные языковые модели не работали бы. Но *CE* не полностью совпадает с тем, что мы хотим от финальной системы.

22.2 *Exposure bias*

При *teacher forcing* модель на шаге t получает правильный *prefix* $y_{<t}$. На *inference* она получает собственный *prefix* $\hat{y}_{<t}$. Если в начале была ошибка, модель может оказаться в области *prefix*'ов, которые редко видела при обучении. Ошибка начинает самоподдерживаться.

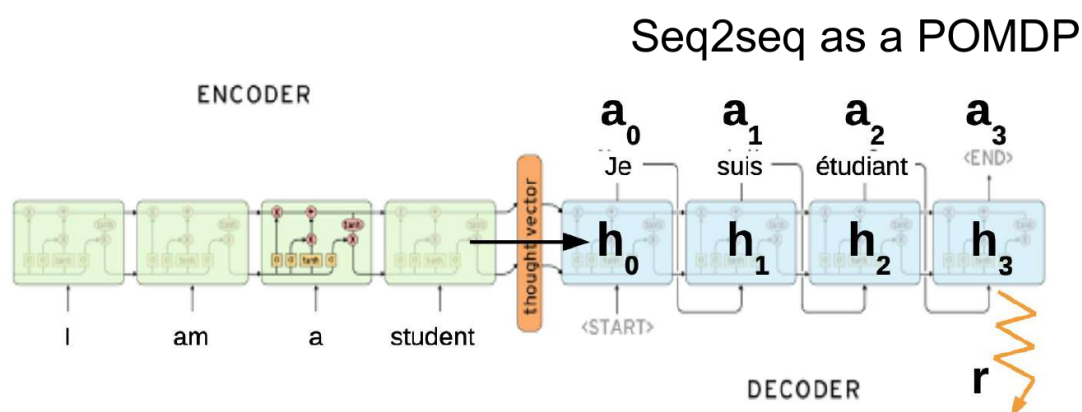
22.3 *Token-level objective* против *sequence-level quality*

CE штрафует каждый токен независимо в рамках заданного *reference*. Но качество ответа часто *sequence-level*: перевод может быть хорошим даже с другими словами; *summary* может быть фактически неверным из-за одной сущности; диалоговый ответ может быть грамматичным, но бесполезным. *BLEU*, *human preference*, *factuality* и *task success* не являются тем же самым, что *token likelihood*.

22.4 *One-to-many* и *mode covering*

Для одного *context* существует много правильных продолжений. *MLE* стремится покрыть распределение данных. В открытой генерации это может приводить к усредненным, безопасным, но скучным ответам. *Decoding* частично решает это через *sampling*, *temperature*, *top-k/top-p*, *beam search*, но *decoding* не меняет сам *training objective*.

22.5 Почему здесь появляется RL



Hidden state $\mathbf{s}$ = translation/conversation state

Initial state $\mathbf{s}$ = encoder output

Observation $\mathbf{o}$ = previous words

Action $\mathbf{a}$ = write next word

Reward $\mathbf{r}$ = domain-specific reward (e.g. BLEU)

Рис. 61: Seq2seq как POMDP. Для генерации state можно понимать как encoder output и уже сгенерированный prefix, action - следующий token, reward - внешняя метрика вроде BLEU/CIDEr или пользовательская оценка.

Self-critical sequence training

Idea: use inference mode as a baseline!

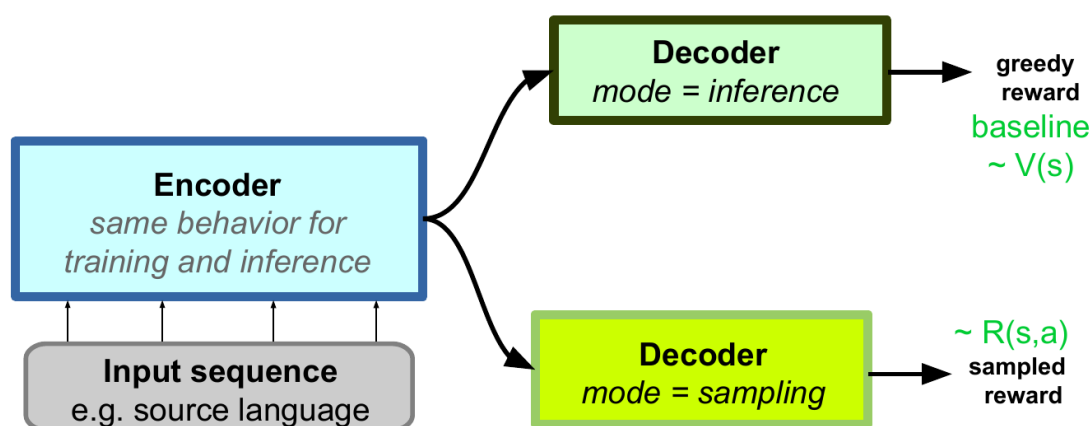


Рис. 62: Self-critical sequence training. SCST использует reward greedy inference как baseline и reward sampled output как обучающий сигнал. Это снижает дисперсию и оптимизирует sequence-level metric.

Если есть reward model или task-specific reward, можно оптимизировать sequence-level objective через RL. Тогда модель получает сигнал не только за совпадение с reference токенами, а за качество целого ответа. Это связано с policy gradient: языковая модель рассматривается как policy, которая выбирает токены, а reward оценивает сгенерированную последовательность.

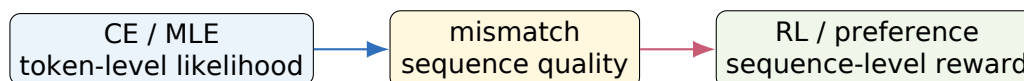


Рис. 63: CE остается базой, но для некоторых требований нужен дополнительный sequence-level сигнал.

Как отвечать на экзамене

Cross-entropy/MLE обучает модель предсказывать следующий токен из training data, и это фундаментальная objective для language modeling. Но ее не всегда достаточно: есть exposure bias из-за teacher forcing, mismatch между token-level loss и sequence-level quality, много правильных ответов для одного context, проблемы factuality/preference/safety и зависимость от decoding. Поэтому после MLE могут использовать RL или preference optimization, где языковая модель рассматривается как policy, а reward задает качество целой последовательности.

Частые ошибки

Не надо говорить, что CE “не работает”. Она работает очень хорошо и является основой pretraining. Корректная формулировка: CE не полностью совпадает с финальной пользовательской или task-specific метрикой, поэтому ее часто дополняют другими этапами обучения и оценки.

Финальная шпаргалка: связи между темами

- BoW/TF-IDF - ручная статистика слов. Word2vec - обучаемая статистика контекстов.
- Negative sampling и contrastive learning связаны идеей “positive vs negatives”.
- RNN решает последовательности через hidden state; attention убирает bottleneck одного состояния.
- Self-attention - attention внутри одной последовательности; Transformer - стек self-attention + FFN + residual + norm.
- BERT - encoder-only Transformer с MLM/NSP pretraining; fine-tuning - transfer learning на downstream задачи.
- CLIP - contrastive learning для image-text пар; zero-shot работает через текстовые prompts.
- RL отличается от supervised тем, что данные зависят от действий policy, а reward может быть delayed.
- Value оценивает состояние, Q оценивает пару state-action.
- Policy gradient увеличивает вероятность действий, приведших к высокому return; baseline уменьшает variance.
- CE оптимизирует token likelihood, а RL/preference методы могут оптимизировать sequence-level reward.

Типовые устные вопросы

1. **Почему TF-IDF лучше count BoW?** Потому что он штрафует слова, встречающиеся почти во всех документах, и выделяет специфичные термины.
2. **Почему Word2vec учит семантику без разметки?** Потому что контекстная prediction task заставляет похожие слова иметь похожие векторы.
3. **Зачем делить attention scores на $\sqrt{d_k}$?** Чтобы стабилизировать дисперсию dot products и не насыщать softmax.
4. **Почему Transformer лучше параллелится, чем RNN?** Потому что все токены слоя self-attention можно обрабатывать одновременно, а RNN зависит от предыдущего hidden state.
5. **Почему BERT не causal LM?** Он bidirectional encoder и предсказывает masked токены по левому и правому контексту.
6. **Как CLIP делает zero-shot?** Классы записываются текстовыми prompts; выбирается prompt с максимальной similarity к image embedding.
7. **Почему baseline не меняет policy gradient?** Его вклад имеет нулевое матожидание, если он не зависит от action.
8. **Чем V отличается от Q ?** $V(s)$ оценивает состояние, $Q(s, a)$ оценивает действие в этом состоянии.

План подготовки на 3 дня

День 1. Классическое NLP и последовательности. BoW, TF-IDF, Word2vec, negative sampling, RNN, seq2seq, attention. Цель дня - уметь нарисовать pipeline от текста к векторам и объяснить bottleneck RNN encoder-decoder.

День 2. Transformers и representation learning. Self-attention, Transformer, BERT pretraining/fine-tuning, contrastive learning, CLIP, evaluation generation. Цель дня - уверенно различать encoder-only, decoder-only, encoder-decoder и объяснять QKV.

День 3. RL и связь с language modeling. MDP, Markov property, return, value, Q, MC/TD, policy gradient, baseline, CE limitations. Цель дня - вывести policy gradient estimator и объяснить, зачем RL может понадобиться после CE.

Приложение А. Сквозной пример: от сырого текста к модели

В основных билетах методы рассматривались отдельно. На экзамене полезно уметь связать их в один практический pipeline. Рассмотрим задачу sentiment classification: по отзыву нужно предсказать, положительный он или отрицательный.

Шаг 1. Формулируем задачу

Есть обучающая выборка

$$\mathcal{D} = \{(d_i, y_i)\}_{i=1}^n, \quad y_i \in \{0, 1\}.$$

Документ d_i - текст отзыва, label y_i - тональность. Сначала нужно выбрать способ представить текст в числах, затем выбрать модель и loss. Уже здесь видно отличие NLP от табличного ML: признаки не даны явно, их нужно построить из последовательности токенов.

Шаг 2. Baseline через BoW/TF-IDF

Пусть после токенизации и нормализации получили словарь:

$$\mathcal{V} = \{\text{good, bad, not, movie, boring}\}.$$

Тогда отзывы можно представить матрицей document-term:

Документ	good	bad	not	movie	boring
"good movie"	1	0	0	1	0
"bad boring movie"	0	1	0	1	1
"not good"	1	0	1	0	0

Для маленького примера BoW уже показывает проблему: "good" в "not good" выглядит положительно, если не использовать n-grams. Поэтому добавление биграмм вроде "not good" может резко улучшить линейную модель.

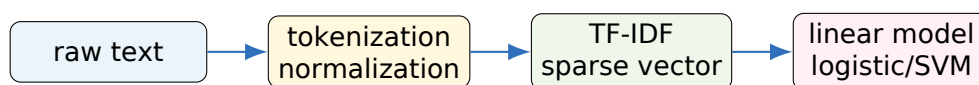


Рис. 64: Классический NLP pipeline для простого baseline.

Такой baseline хорош тем, что он быстро обучается и интерпретируем. Если вес при слове "excellent" положительный, а при "awful" отрицательный, это легко объяснить. Но он плохо обобщает синонимы и редко понимает композицию смысла.

Шаг 3. Word embeddings

Если заменить one-hot слова на embeddings, модель начинает использовать близость слов. Например, "excellent" и "great" могут иметь близкие векторы, даже если одно из слов редко встречалось в обучающей выборке. Но простой average of word embeddings все еще может потерять порядок и отрицания. Поэтому embeddings являются не финальным ответом, а более богатым строительным блоком.

Шаг 4. RNN и Transformer

RNN читает отзыв последовательно и может, по крайней мере теоретически, учитывать порядок слов. Фраза “not good” может дать другое hidden state, чем “good”. Transformer делает это через self-attention: токен “good” может обратить внимание на “not”, а классификационный токен может агрегировать важные слова со всего текста.

Полезное замечание

На практике разумный порядок экспериментов часто такой: TF-IDF + linear model как baseline; затем pretrained BERT-like encoder для качества; затем анализ ошибок. Если baseline уже решает задачу почти идеально, сложная модель может быть не нужна.

Приложение В. Размерности тензоров, которые любят спрашивать

Многие устные вопросы проверяют не только понимание идеи, но и умение отслеживать shapes. Ниже собраны типичные размерности.

BoW и TF-IDF

Если в корпусе N документов, словарь размера $|\mathcal{V}| = m$, то document-term matrix имеет размер $N \times m$. Строка - документ, столбец - токен или n-gram. Векторы обычно sparse.

Word2vec

Объект	Размер	Смысл
W_{in}	$ \mathcal{V} \times d$	input embeddings
W_{out}	$ \mathcal{V} \times d$	output embeddings
v_w	d	embedding центрального слова
u_c	d	embedding контекстного слова
$u_c^\top v_w$	scalar	совместимость пары

Если используется full softmax, logits имеют длину $|\mathcal{V}|$. Если negative sampling с k негативами, считаются только $k + 1$ скалярных произведений.

RNN

Пусть batch size B , длина T , размер embedding d_x , hidden size d_h . Тогда вход имеет размер $B \times T \times d_x$. Hidden states имеют размер $B \times T \times d_h$, а последний hidden state - $B \times d_h$. Матрицы vanilla RNN:

$$W_x \in \mathbb{R}^{d_h \times d_x}, \quad W_h \in \mathbb{R}^{d_h \times d_h}, \quad b \in \mathbb{R}^{d_h}.$$

Количество параметров без output layer:

$$d_h d_x + d_h^2 + d_h.$$

Attention

Пусть $X \in \mathbb{R}^{B \times T \times d_{model}}$, число heads h , размер одной head $d_k = d_{model}/h$. Тогда для одной head:

$$Q, K, V \in \mathbb{R}^{B \times T \times d_k}, \quad QK^\top \in \mathbb{R}^{B \times T \times T}.$$

После умножения на V снова получается $B \times T \times d_k$. После concatenation всех heads - $B \times T \times d_{model}$.

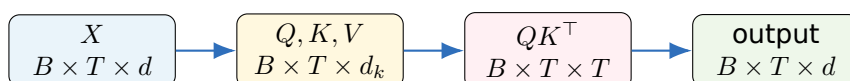


Рис. 65: Главный shape self-attention - квадратичная матрица $T \times T$.

BERT fine-tuning

Если sequence length T , hidden size d , число классов C , то BERT output $H \in \mathbb{R}^{B \times T \times d}$. Для sentence classification берут $H_{[:,0,:]} \in \mathbb{R}^{B \times d}$ и получают logits $B \times C$. Для token classification линейная head применяется к каждому токеноу и дает $B \times T \times C$.

RL trajectories

Trajectory можно записать как

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T).$$

Return G_t - scalar для каждого шага. Value $V(s_t)$ - scalar. Q-values для дискретных действий часто представлены вектором длины $|\mathcal{A}|$: $Q(s_t, \cdot) \in \mathbb{R}^{|\mathcal{A}|}$.

Приложение С. Мини-выводы, которые стоит уметь повторить

С.1. Почему MLE для language modeling дает cross-entropy

Пусть обучающий текст $y_1, \dots, y_T$ разложен авторегрессивно:

$$p_\theta(y_1, \dots, y_T) = \prod_{t=1}^T p_\theta(y_t | y_{<t}).$$

MLE максимизирует log-likelihood:

$$\log p_\theta(y) = \sum_{t=1}^T \log p_\theta(y_t | y_{<t}).$$

Минимизация отрицательного log-likelihood дает

$$\mathcal{L}_{NLL} = - \sum_{t=1}^T \log p_\theta(y_t | y_{<t}),$$

что совпадает с token-level cross-entropy для one-hot target distribution. Это объясняет, почему CE так естественна для языкового моделирования.

С.2. Почему softmax attention является выпуклой комбинацией values

Softmax дает неотрицательные веса:

$$\alpha_i = \frac{e^{e_i}}{\sum_j e^{e_j}} \geq 0.$$

Их сумма равна единице:

$$\sum_i \alpha_i = 1.$$

Поэтому context vector

$$c = \sum_i \alpha_i v_i$$

лежит в выпуклой оболочке value-векторов. Это полезная интуиция: attention не выбирает один элемент жестко, а дифференцируемо смешивает информацию.

С.3. Беллмановская рекурсия для value

По определению

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s].$$

Так как $G_t = r_t + \gamma G_{t+1}$, получаем

$$V^\pi(s) = \mathbb{E}_\pi[r_t + \gamma G_{t+1} | S_t = s].$$

Но условное ожидание будущего return из S_{t+1} - это $V^\pi(S_{t+1})$, поэтому

$$V^\pi(s) = \mathbb{E}_\pi[r_t + \gamma V^\pi(S_{t+1}) | S_t = s].$$

Это и есть Bellman expectation equation.

С.4. Bellman optimality для Q

Оптимальное действие после перехода выбирается по максимуму:

$$Q^*(s, a) = \mathbb{E}[r_t + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a].$$

Q-learning пытается приблизить эту рекурсию сэмпловыми обновлениями. В deep Q-learning нейросеть обучается минимизировать squared TD error:

$$\left(Q_{\theta}(s, a) - \left(r + \gamma \max_{a'} Q_{\theta-}(s', a') \right) \right)^2.$$

С.5. Полный вывод baseline-unbiasedness

Для любого baseline $b(s)$, не зависящего от action:

$$\mathbb{E}_{a \sim \pi(\cdot | s)}[\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] = b(s) \sum_a \pi(a | s) \frac{\nabla_{\theta} \pi(a | s)}{\pi(a | s)}.$$

Сокращая π , получаем

$$b(s) \sum_a \nabla_{\theta} \pi(a | s) = b(s) \nabla_{\theta} \sum_a \pi(a | s) = b(s) \nabla_{\theta} 1 = 0.$$

Значит, baseline не меняет математическое ожидание policy gradient, но может уменьшить variance. Это один из самых частых коротких выводов на экзамене.

Приложение D. Экзаменационные мини-задачи с решениями

1. Посчитать TF-IDF на игрушечном корпусе

Пусть $N = 10$, слово встретилось в документе 3 раза, длина документа 100 токенов, а $df(t) = 2$. При нормированном TF:

$$tf = 3/100 = 0.03.$$

Сглаженный IDF:

$$idf = \log \frac{11}{3} + 1.$$

Итоговый вес $tfidf = 0.03(\log(11/3) + 1)$. В ответе важно проговорить: TF считается внутри документа, IDF - по корпусу документов.

2. Почему bigram features помогают с отрицаниями?

Unigram BoW видит “not” и “good” отдельно. Если “good” обычно положительное, линейная модель может ошибиться на “not good”. Биграмма “not good” становится отдельным признаком и может получить отрицательный вес. Это простой способ вернуть часть локального порядка без перехода к RNN/Transformer.

3. Что будет, если в negative sampling брать негативы равномерно?

Модель будет слишком часто видеть редкие слова как negatives и слишком редко - частые. Это может ухудшить качество embeddings, потому что noise distribution перестанет отражать реальную частотность языка. Распределение $f(w)^{3/4}$ является компромиссом между частотностью и сглаживанием.

4. Почему RNN трудно учить дальние зависимости?

Градиент к ранним токенам проходит через длинную цепочку умножений. Если нормы соответствующих якобианов меньше единицы, градиент затухает; если больше - взрывается. Поэтому vanilla RNN плохо хранит long-range dependencies, а LSTM/GRU добавляют gated memory paths.

5. Почему attention помогает переводу длинных предложений?

Без attention encoder должен сжать весь source в один vector summary. Attention хранит все encoder states и позволяет decoder на каждом шаге строить отдельный context vector. Поэтому при генерации каждого target token модель может использовать релевантные source positions.

6. Чем beam search отличается от greedy decoding?

Greedy хранит одну гипотезу и на каждом шаге выбирает самый вероятный следующий токен. Beam search хранит B лучших partial hypotheses, расширяет их и оставляет лучшие. Beam search дороже, но может найти последовательность с лучшей суммарной log-probability.

7. Почему BLEU может быть низким для хорошего перевода?

BLEU основан на n -gram overlap с reference. Если candidate является хорошей парфразой, но использует другие слова и порядок, overlap может быть низким. Поэтому BLEU полезен как corpus-level automatic metric, но не является абсолютной мерой качества одного перевода.

8. Почему unsupervised MT нуждается в back-translation?

Denoising autoencoder учит восстанавливать предложения внутри языка, но не создает прямого соответствия между языками. Back-translation генерирует synthetic parallel pairs: предложение на одном языке переводится текущей обратной моделью, после чего пара используется для обучения прямого направления.

9. Почему self-attention требует positional encoding?

Без positional information self-attention видит набор token representations и попарные similarities, но не знает исходный порядок. Если переставить токены и так же переставить входные строки, механизм даст соответствующе переставленный результат. Для языка порядок критичен, поэтому добавляют positional encodings.

10. Что означает masked self-attention в decoder?

Это causal mask, запрещающая позиции t смотреть на будущие токены $> t$. Иначе при обучении decoder мог бы видеть правильный следующий token и задача language modeling стала бы тривиальной и нечестной.

11. Зачем Transformer FFN, если уже есть attention?

Attention смешивает информацию между позициями. FFN применяет нелинейное преобразование к представлению каждой позиции отдельно. Без FFN блок был бы менее выразительным: он умел бы агрегировать токены, но хуже преобразовывал бы features внутри позиции.

12. Почему BERT использует MLM вместо left-to-right LM?

BERT - bidirectional encoder: token может смотреть и налево, и направо. Если обучать обычную LM, модель могла бы увидеть предсказываемый token. MLM скрывает часть токенов и заставляет восстанавливать их по контексту.

13. Когда frozen BERT может быть лучше full fine-tuning?

На очень маленьких данных full fine-tuning может переобучиться или разрушить pretrained weights. Frozen encoder дешевле и стабильнее, хотя часто дает меньший максимум качества. Это хороший вариант для быстрого baseline или ограниченной памяти.

14. Что такое in-batch negatives?

Если batch содержит пары (x_i, y_i) , то для anchor x_i positive - y_i , а все остальные y_j в том же batch считаются negatives. Это дает много негативов без отдельного sampling и используется в CLIP-like обучении.

15. Почему CLIP может классифицировать классы, которых не было как labels?

CLIP не обучается на фиксированный набор labels. Он учится сопоставлять изображения и тексты. Поэтому новый класс можно задать текстовым prompt, получить его embedding и сравнить с image embedding.

16. Что именно является policy в language-model-as-RL постановке?

Policy - это распределение следующего токена по prefix: $\pi_\theta(a_t | s_t)$, где state можно понимать как prompt и уже сгенерированный prefix, а action - выбор следующего токена. Episode - генерация всей последовательности.

17. Почему Markov property может нарушиться в игре по одному кадру?

Один кадр может не содержать скорость объектов. Два разных прошлых движения могут привести к одинаковой картинке сейчас, но требовать разных действий. Добавление нескольких кадров или recurrent state помогает восстановить скрытую информацию.

18. Как быстро посчитать returns в episode?

Идти с конца назад: положить $G_T = 0$, затем для $t = T - 1, \dots, 0$ считать $G_t = r_t + \gamma G_{t+1}$. Это линейный алгоритм по длине episode.

19. Почему Q-function удобнее value для выбора действия?

$V(s)$ говорит, насколько хорошо состояние при policy, но не сравнивает действия напрямую. $Q(s, a)$ оценивает каждое действие в состоянии, поэтому можно выбрать $\arg \max_a Q(s, a)$.

20. Почему policy gradient имеет высокую дисперсию?

Оценка строится по случайным trajectories. Return зависит от множества будущих случайностей, а одно действие получает кредит за весь последующий результат. Baseline, advantage estimates, actor-critic и reward normalization уменьшают variance.

21. Почему baseline нельзя делать зависящим от action?

Доказательство нулевого вклада использует $\sum_a \nabla \pi(a | s) = 0$. Если baseline зависит от a , его нельзя вынести из суммы, и вклад в expectation может стать ненулевым, то есть estimator станет biased.

22. Почему CE и RL обычно не противопоставляют, а комбинируют?

CE дает сильную языковую модель и учит распределение текста. RL или preference optimization без хорошей начальной модели было бы нестабильным и дорогим. Поэтому сначала делают supervised/pretraining через CE, а затем при необходимости дооптимизируют sequence-level reward.

Приложение Е. Подробные математические выводы и доказательства

Это приложение добавлено специально для того, чтобы формулы из основной части не выглядели как магия. Здесь показано, откуда берутся основные objectives, рекурсии и градиентные оценки. На экзамене не всегда нужно воспроизводить все доказательства полностью, но полезно понимать, какой шаг является определением, какой - предположением, а какой - алгебраическим преобразованием.

Е.1. TF-IDF: почему множитель IDF имеет именно такой смысл

Пусть корпус состоит из N документов. Для термина t обозначим через $df(t)$ число документов, в которых он встречается хотя бы один раз. Если слово встречается почти в каждом документе, то $df(t) \approx N$, и

$$idf(t) = \log \frac{N}{df(t)} \approx \log 1 = 0.$$

Такой терм почти не помогает отличать документы друг от друга. Если же слово встречается в малой доле документов, например $df(t) = N/100$, то

$$idf(t) = \log 100,$$

и слово получает большой вес.

Вывод / доказательство

Можно вывести IDF как количество информации о событии. Пусть случайно выбранный документ содержит слово t с вероятностью

$$p(t) = \frac{df(t)}{N}.$$

В теории информации неожиданность события равна $-\log p(t)$. Тогда

$$-\log p(t) = -\log \frac{df(t)}{N} = \log \frac{N}{df(t)}.$$

Это ровно IDF. Поэтому IDF можно понимать как меру неожиданности появления слова в документе. Чем реже слово встречается по документам, тем больше информации оно несет для различения документов.

Сглаженная версия

$$idf(t) = \log \frac{N+1}{df(t)+1} + 1$$

нужна по практическим причинам: не делить на ноль для новых/редких слов и не получать нулевой вес для слов, встречающихся во всех документах, если в реализации хочется сохранить небольшой вклад TF.

Е.2. Skip-gram: от условной вероятности к cross-entropy loss

В skip-gram есть центральное слово w и контекстное слово c . Для каждого слова есть input embedding v_w и output embedding u_c . Сначала задается score совместимости:

$$s(w, c) = u_c^\top v_w.$$

Чтобы превратить scores по всем словам словаря в вероятности, применяется softmax:

$$p_{\theta}(c | w) = \frac{\exp(u_c^{\top} v_w)}{\sum_{j \in \mathcal{V}} \exp(u_j^{\top} v_w)}.$$

Если корпус дал нам пары (w, c) , то метод максимального правдоподобия максимизирует вероятность всех наблюдаемых пар:

$$L(\theta) = \prod_{(w,c) \in D} p_{\theta}(c | w).$$

Логарифм монотонен, поэтому максимум не меняется:

$$\log L(\theta) = \sum_{(w,c) \in D} \log p_{\theta}(c | w).$$

В оптимизации обычно минимизируют loss, поэтому берут минус:

$$\mathcal{L}(\theta) = - \sum_{(w,c) \in D} \log p_{\theta}(c | w).$$

Вывод / доказательство

Для одного обучающего примера правильный класс - контекстное слово c . Его one-hot label y имеет $y_c = 1$, $y_j = 0$ для $j \neq c$. Cross-entropy между label y и предсказанием p равна

$$H(y, p) = - \sum_{j \in \mathcal{V}} y_j \log p_j = - \log p_c.$$

Значит negative log-likelihood для skip-gram с softmax совпадает с cross-entropy классификацией контекстного слова по центральному слову.

Е.3. Почему полный softmax дорогой

Для одной пары (w, c) нужно посчитать знаменатель

$$Z_w = \sum_{j \in \mathcal{V}} \exp(u_j^{\top} v_w).$$

Это требует пройти по всем словам словаря. Если $|\mathcal{V}| = 500000$, а embedding dimension $d = 300$, то один forward для одного центрального слова требует порядка $|\mathcal{V}| \cdot d$ операций только на dot products. Именно поэтому в Word2vec используют приближения: hierarchical softmax, subsampling frequent words и negative sampling.

Е.4. Negative sampling как бинарная логистическая регрессия

В negative sampling мы больше не спрашиваем: “какое слово из всего словаря является контекстом?”. Мы спрашиваем: “является ли пара (w, c) настоящей парой из корпуса?”. Для положительной пары label равен 1, для случайной пары (w, n) label равен 0.

Вероятность того, что пара настоящая, задается через sigmoid:

$$P(y = 1 | w, c) = \sigma(u_c^{\top} v_w), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Для label $y \in \{0, 1\}$ бинарная log-likelihood:

$$\log P(y | z) = y \log \sigma(z) + (1 - y) \log(1 - \sigma(z)).$$

Так как $1 - \sigma(z) = \sigma(-z)$, для positive pair получаем $\log \sigma(u_c^\top v_w)$, а для negative pair (w, n) получаем $\log \sigma(-u_n^\top v_w)$. Поэтому objective для одной положительной пары и k негативов:

$$\log \sigma(u_c^\top v_w) + \sum_{i=1}^k \log \sigma(-u_{n_i}^\top v_w).$$

Loss для минимизации - это минус этой величины.

Вывод / доказательство

Посмотрим на градиент по score z для положительной пары. Loss $\ell_+(z) = -\log \sigma(z)$. Тогда

$$\frac{d\ell_+}{dz} = -(1 - \sigma(z)) = \sigma(z) - 1.$$

Если z маленький, $\sigma(z) \approx 0$, градиент отрицательный, и градиентный спуск увеличивает z . Для негативной пары $\ell_-(z) = -\log \sigma(-z)$. Тогда

$$\frac{d\ell_-}{dz} = \sigma(z).$$

Градиентный спуск уменьшает z . Значит positive pairs получают большие dot products, negatives - маленькие. Это и есть геометрический смысл negative sampling.

E.5. RNN: от рекуррентной формулы к vanishing gradients

Обычная RNN задается так:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b).$$

Пусть loss зависит от финального состояния h_T . Чтобы понять, как h_t влияет на h_T , применим chain rule:

$$\frac{\partial h_T}{\partial h_t} = \frac{\partial h_T}{\partial h_{T-1}} \frac{\partial h_{T-1}}{\partial h_{T-2}} \dots \frac{\partial h_{t+1}}{\partial h_t}.$$

Каждый множитель имеет вид

$$\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(\phi'(a_k)) W_h, \quad a_k = W_x x_k + W_h h_{k-1} + b.$$

Вывод / доказательство

Если нормы этих Якобианов в среднем меньше 1, то норма произведения убывает примерно экспоненциально по длине цепочки:

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{k=t+1}^T \left\| \frac{\partial h_k}{\partial h_{k-1}} \right\|.$$

Если каждый множитель примерно $\rho < 1$, то получаем ρ^{T-t} . Это vanishing gradient. Если множители больше 1, получается exploding gradient. Поэтому длинные зависимости плохо учатся обычной RNN.

LSTM и GRU добавляют gates и более прямые пути переноса информации, чтобы градиент мог проходить через время стабильнее.

E.6. Seq2seq likelihood: почему loss является суммой по токенам

Задача машинного перевода: по source sequence x сгенерировать target sequence $y = (y_1, \dots, y_T)$. По правилу цепочки вероятностей:

$$p(y | x) = p(y_1, \dots, y_T | x) = \prod_{t=1}^T p(y_t | y_1, \dots, y_{t-1}, x).$$

Сокращенно:

$$p(y | x) = \prod_{t=1}^T p(y_t | y_{<t}, x).$$

Максимизация likelihood по параллельному корпусу эквивалентна минимизации negative log-likelihood:

$$\mathcal{L} = -\log p(y | x) = -\sum_{t=1}^T \log p(y_t | y_{<t}, x).$$

Именно эта формула реализуется как token-level cross-entropy с teacher forcing: на шаге t decoder получает истинный prefix $y_{<t}$.

E.7. Beam search: почему используют log-probability и length normalization

Вероятность гипотезы $y_{1:T}$ равна произведению токенных вероятностей:

$$p(y_{1:T} | x) = \prod_{t=1}^T p(y_t | y_{<t}, x).$$

Произведение многих чисел из $(0, 1)$ быстро становится очень маленьким, поэтому работают с логарифмом:

$$\log p(y_{1:T} | x) = \sum_{t=1}^T \log p(y_t | y_{<t}, x).$$

Но сумма логарифмов почти всегда уменьшается при увеличении длины, поэтому без нормировки beam search склонен предпочитать слишком короткие гипотезы. Простая коррекция:

$$score(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log p(y_t | y_{<t}, x),$$

где α - гиперпараметр length normalization.

E.8. Attention: почему output - выпуклая комбинация values

Attention scores e_i превращаются в веса через softmax:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}.$$

Так как $\exp(e_i) > 0$, все $\alpha_i > 0$. Кроме того,

$$\sum_i \alpha_i = \sum_i \frac{\exp(e_i)}{\sum_j \exp(e_j)} = 1.$$

Attention output:

$$c = \sum_i \alpha_i v_i.$$

Это выпуклая комбинация vectors v_i : модель не обязана выбирать один value, она дифференцируемо смешивает информацию из всех позиций.

Вывод / доказательство

Почему делят на $\sqrt{d_k}$? Пусть координаты query и key независимы, имеют нулевое среднее и дисперсию 1. Тогда dot product

$$q^\top k = \sum_{r=1}^{d_k} q_r k_r.$$

У каждого произведения $q_r k_r$ среднее 0 и дисперсия 1. Для суммы дисперсия равна d_k . Значит типичный масштаб dot product растет как $\sqrt{d_k}$. Деление на $\sqrt{d_k}$ возвращает scores к масштабу порядка 1 и не дает softmax преждевременно насыщаться.

E.9. Self-attention: все размерности

Пусть sequence length T , hidden size d_{model} . Входной тензор одного примера:

$$X \in \mathbb{R}^{T \times d_{model}}.$$

Для одной головы:

$$W_Q, W_K \in \mathbb{R}^{d_{model} \times d_k}, \quad W_V \in \mathbb{R}^{d_{model} \times d_v}.$$

Тогда

$$Q = XW_Q \in \mathbb{R}^{T \times d_k}, \quad K = XW_K \in \mathbb{R}^{T \times d_k}, \quad V = XW_V \in \mathbb{R}^{T \times d_v}.$$

Матрица scores:

$$\frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

После softmax по строкам получаем attention matrix $A \in \mathbb{R}^{T \times T}$. Output:

$$AV \in \mathbb{R}^{T \times d_v}.$$

Для multi-head attention несколько таких outputs конкатенируются, а затем умножаются на W_O , чтобы вернуться к размерности d_{model} .

E.10. LayerNorm: что именно нормируется

Для одного токена есть hidden vector $h \in \mathbb{R}^d$. LayerNorm считает среднее и дисперсию по координатам этого вектора:

$$\mu = \frac{1}{d} \sum_{i=1}^d h_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (h_i - \mu)^2.$$

Затем нормирует:

$$\text{LN}(h)_i = \gamma_i \frac{h_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

Важное отличие от BatchNorm: LayerNorm не требует статистик по batch, поэтому хорошо подходит для последовательностей, autoregressive decoding и переменной длины.

E.11. BERT MLM objective

Пусть входная последовательность $x = (x_1, \dots, x_T)$, а M - множество выбранных masked positions. BERT получает испорченный вход $\tilde{x}$ и предсказывает исходные токены только на позициях M :

$$\mathcal{L}_{MLM} = - \sum_{i \in M} \log p_{\theta}(x_i | \tilde{x}).$$

Почему loss не считается по всем позициям? Потому что задача именно восстановить скрытые токены по контексту. Если считать loss по неизмененным токенам как обычное копирование, модель получила бы слишком много тривиального сигнала.

E.12. InfoNCE как обычная cross-entropy

Есть anchor x , positive x^+ и $N-1$ negatives $x_1^-, \dots, x_{N-1}^-$. Обозначим кандидатов $z_1, \dots, z_N$, где $z_1 = x^+$, а остальные - negatives. Score:

$$s_i = \text{sim}(f(x), f(z_i)) / \tau.$$

Вероятность, что кандидат i является positive:

$$p(i | x) = \frac{e^{s_i}}{\sum_{j=1}^N e^{s_j}}.$$

Если правильный индекс 1, то cross-entropy:

$$\mathcal{L} = -\log p(1 | x) = -\log \frac{e^{s_1}}{\sum_{j=1}^N e^{s_j}}.$$

Это и есть InfoNCE. Поэтому contrastive learning можно объяснять как задачу классификации: найти правильную пару среди набора кандидатов.

E.13. CLIP loss: две симметричные классификации

В batch есть N пар изображение-текст (I_i, T_i) . Image encoder дает нормированные embeddings u_i , text encoder - v_i . Матрица similarities:

$$S_{ij} = \frac{u_i^\top v_j}{\tau}.$$

Для каждого изображения правильный текст имеет тот же индекс i :

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp S_{ii}}{\sum_{j=1}^N \exp S_{ij}}.$$

Симметрично, для каждого текста правильное изображение:

$$\mathcal{L}_{T \rightarrow I} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp S_{ii}}{\sum_{j=1}^N \exp S_{ji}}.$$

Итог:

$$\mathcal{L}_{CLIP} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I}).$$

Так модель одновременно учится искать текст по изображению и изображение по тексту.

E.14. Discounted return: рекурсия и ограниченность

Определение discounted return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Вынесем первый reward:

$$G_t = r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots) = r_t + \gamma G_{t+1}.$$

Вывод / доказательство

Если $|r_t| \leq R_{max}$ и $0 \leq \gamma < 1$, то

$$|G_t| \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = R_{max} \frac{1}{1-\gamma}.$$

Значит discounted return конечен даже для бесконечного горизонта. Но важно помнить: discounting не просто технический прием, он меняет оптимизационную задачу, потому что дальние награды становятся менее важными.

E.15. Bellman expectation equation для V^π

По определению

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Используем рекурсию return:

$$G_t = R_t + \gamma G_{t+1}.$$

Тогда

$$V^\pi(s) = \mathbb{E}_\pi[R_t + \gamma G_{t+1} \mid S_t = s].$$

Теперь раскрываем expectation по action $a \sim \pi(\cdot \mid s)$ и next state $s' \sim P(\cdot \mid s, a)$:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) (r(s, a, s') + \gamma V^\pi(s')).$$

Это Bellman expectation equation: value текущего состояния равен ожидаемой immediate reward плюс discounted value следующего состояния.

E.16. Bellman equation для Q^π и связь V/Q

Определение:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Первое действие зафиксировано, поэтому policy stochasticity начинается только со следующего состояния:

$$Q^\pi(s, a) = \sum_{s'} P(s' \mid s, a) (r(s, a, s') + \gamma V^\pi(s')).$$

А V^π выражается через Q^π усреднением по действиям policy:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a).$$

Если подставить второе в первое, получим рекурсию только через Q :

$$Q^\pi(s, a) = \sum_{s'} P(s' \mid s, a) \left(r(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right).$$

E.17. Bellman optimality и Q-learning target

Оптимальная value function выбирает лучшее действие:

$$V^*(s) = \max_a Q^*(s, a).$$

Поэтому

$$Q^*(s, a) = \mathbb{E} \left[R_t + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

Когда модель dynamics неизвестна, мы заменяем expectation одним sampled transition (s, a, r, s') и строим target:

$$y = r + \gamma \max_{a'} Q(s', a').$$

Табличное Q-learning обновление:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (y - Q(s, a)).$$

Если Q - нейросеть Q_θ , минимизируем squared TD error:

$$\mathcal{L}(\theta) = \left(Q_\theta(s, a) - \left[r + \gamma \max_{a'} Q_\theta(s', a') \right] \right)^2.$$

Параметры θ^- часто замораживают в target network, чтобы цель не двигалась одновременно с обучаемой сетью.

E.18. Monte Carlo и TD: формальная разница

Monte Carlo estimator использует полный return:

$$\hat{Q}_{MC}(s, a) = \frac{1}{N(s, a)} \sum_{i: (s, a) \in \tau_i} G_t^{(i)}.$$

Он не делает bootstrap, но требует дождаться конца episode и может иметь высокую дисперсию.

TD estimator использует один шаг и текущую оценку будущего:

$$y_{TD} = r + \gamma V(s') \quad \text{или} \quad y_{TD} = r + \gamma \max_{a'} Q(s', a').$$

Он biased, если текущая value function неточна, но обычно имеет меньшую дисперсию и учится по неполным trajectories.

E.19. Policy gradient theorem: вывод через распределение trajectory

Пусть trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ имеет вероятность

$$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Objective:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [R(\tau)] = \int p_\theta(\tau) R(\tau) d\tau.$$

Дифференцируем:

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) R(\tau) d\tau.$$

Используем log-derivative trick:

$$\nabla p_{\theta}(\tau) = p_{\theta}(\tau) \nabla \log p_{\theta}(\tau).$$

Тогда

$$\nabla_{\theta} J(\theta) = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)].$$

Теперь раскроем логарифм trajectory probability:

$$\log p_{\theta}(\tau) = \log p(s_0) + \sum_t \log \pi_{\theta}(a_t | s_t) + \sum_t \log P(s_{t+1} | s_t, a_t).$$

Среда P и начальное распределение не зависят от θ , поэтому

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Получаем REINFORCE estimator:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right].$$

В более точной версии для каждого шага используют reward-to-go G_t , потому что прошлые reward не зависят от текущего действия:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right].$$

Е.20. Почему baseline не меняет математическое ожидание градиента

Нужно доказать, что для baseline $b(s)$, не зависящего от action,

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] = 0.$$

Действительно:

$$\begin{aligned} \mathbb{E}_a [\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] &= \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) b(s) \\ &= b(s) \sum_a \pi_{\theta}(a | s) \frac{\nabla_{\theta} \pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} \\ &= b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) \\ &= b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) \\ &= b(s) \nabla_{\theta} 1 = 0. \end{aligned}$$

Значит можно заменить G_t на $G_t - b(s_t)$ без bias. Если выбрать $b(s) = V^{\pi}(s)$, множитель становится advantage:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

Интуитивно policy усиливает действия, которые лучше среднего для данного состояния, а не просто действия из хороших состояний.

E.21. Cross-entropy для language modeling: где возникает exposure bias

При teacher forcing оптимизируется

$$\mathcal{L}_{CE} = - \sum_t \log p_{\theta}(y_t^{ref} | y_{<t}^{ref}, x).$$

Модель учится предсказывать следующий токен при условии правильного prefix. На inference она видит свой собственный prefix:

$$p_{\theta}(y_t | \hat{y}_{<t}, x),$$

где $\hat{y}_{<t}$ может содержать ошибки. Поэтому training distribution по hidden states отличается от inference distribution. Это и называют exposure bias.

Еще одна проблема: CE - token-level objective, а качество часто задается sequence-level reward:

$$R(y, x) = BLEU(y, y^{ref}) \quad \text{или} \quad R(y, x) = \text{human preference}.$$

Токен, который немного снижает log-likelihood, может улучшить итоговый sequence reward, и наоборот. Поэтому CE обычно используют для pretraining/supervised training, а RL/preference методы - для донастройки под sequence-level цель.

Последняя страница: 22 ответа в одну строку

1. BoW считает слова, TF-IDF штрафует корпусно-частые слова.
2. Word2vec учит embeddings через предсказание контекста или слова по контексту.
3. Negative sampling заменяет softmax по словарю бинарной классификацией positive/negative pairs.
4. RNN обновляет hidden state по текущему входу и прошлому state.
5. Attention - weighted sum values с весами из query-key similarities.
6. Encoder-decoder моделирует $p(y|x)$ и генерирует target autoregressively.
7. Generation оценивают PPL, BLEU/ROUGE, semantic metrics, human/preference eval.
8. Unsupervised MT использует monolingual corpora, denoising и back-translation.
9. Self-attention строит Q, K, V из одной последовательности.
10. Transformer - MHA + FFN + residual + norm + positions.
11. BERT pretraining = MLM + NSP в оригинальной версии.
12. BERT fine-tuning = pretrained encoder + task head + адаптация.
13. Contrastive learning сближает positives и отдаляет negatives.
14. CLIP учит image/text encoders через batch contrastive loss.
15. RL - агент выбирает действия и максимизирует expected return.
16. Markov property: текущее state достаточно для распределения будущего.
17. Return $G_t = \sum_k \gamma^k r_{t+k}$.
18. $V(s)$ - expected return из состояния; для session оценивается через returns.
19. $Q(s, a)$ - expected return после действия; учится MC или TD/Bellman.
20. Policy gradient использует $\nabla \log \pi(a|s) G_t$.
21. Baseline уменьшает variance и не меняет expectation, если не зависит от action.
22. CE важна, но не равна sequence-level human/task reward.